

TypstQQ 群技术精华

Typst 中文交流群聊天记录提炼 · 问题解答 / 代码效果 / 代码原理

资料来源: E:/group/chunks 历史聊天记录 (约 8500 条)

1 前言

本文档由 Typst 中文交流群 (TypstQQ) 的历史聊天记录提炼而成, 目标是把群里有价值的 技术讨论 沉淀为可检索、可复现的资料。内容按“问题—解答”“代码—效果”“代码—原理”三种形式组织, 群友贴出的 /typ 代码被成对保留。对于能在 Typst 中直接复现的效果, 文档不再插入聊天截图, 而是在代码块后以「▲ 代码实测」实时渲染 (由本机 Typst 在编译时执行 `#eval(read(...))` 生成)。

说明: 聊天记录中的时间戳、发言人 id 等元数据已过滤; 代码块以 raw 形式展示 (不执行), 部分片段依赖较新版本 Typst (如 0.15 的 `math.lr` 基线、`tiling` 等), 运行时请按需升级。文中截图若标注“未包含在本地资源中”, 表示对应的图片文件未随聊天记录一同导出。

目录

1 前言	2
2 第一部分 群聊精选 (早期, seq <= 269536)	5
2.1 列表 / 编号	5
2.2 标题 / 目录	5
2.3 段落 / 中文排版	5
2.4 数学	5
2.5 表格	6
2.6 图形 / 页面	6
2.7 show 规则 / 作用域	6
2.8 字体	6
2.9 参考文献	7
2.10 工具链 / 其他	7
3 第二部分 群聊精选 (2026-06-17 06-24)	8
3.1 行距与基线 (排版原理类)	8
3.2 盒模型 / 布局技巧	8
3.3 数学排版专题	8
3.4 语言 / 数据结构讨论	9
3.5 编号 / 文档结构	9
3.6 FAQ 机器人 (/typ 指令完整说明, part_004 帮助文本)	10
3.7 工具链 / 生态	10
3.8 可嵌入资源 (本轮核实存在)	10
4 第三部分 群聊精选 (2026-06-24 06-27)	11
4.1 数学排版	11
4.2 字体与文字样式	13
4.3 盒模型与布局	15
4.4 表格	16
4.5 语言特性	18
4.6 页面与版式	19
4.7 编号与引用 / 参考文献	23
4.8 图片与绘图	25
4.9 导出与 HTML	26
4.10 坑与 bug / 版本差异	26
4.11 工具链与生态包	27
5 第四部分 群聊精选 (2026-06-27 07-01)	29
5.1 绘图 / 渐变 / 曲线	29
5.2 数学排版	29
5.3 HTML 导出 / 自定义属性	29

5.4	show 规则 / context	29
5.5	引用 / 交叉引用 / 大纲	29
5.6	标题编号 / counter	30
5.7	变长参数	30
5.8	表格	30
5.9	图片导入	30
5.10	列 / 页面	31
5.11	typst eval / 作用域	31
5.12	工具链 / 生态包	31
5.13	中文字体 / 中文社区	31
5.14	文献著录 (GB/T 7714) 深度专题	31
5.15	WYSIWYG 编辑器 / 笔记软件	32
5.16	待补 / 未解	32
6	第五部分 群聊精选 (2026-07-01 前后)	33
6.1	字体与上下标渲染 (Inter super/sub 大案)	33
6.2	数学排版	35
6.3	文本、断行与两端对齐	35
6.4	页面与版式	36
6.5	语言特性 (state / context / 函数)	36
6.6	字典、参数与模板设计	37
6.7	列表与编号	37
6.8	参考文献与 GB/T 7714	38
6.9	工具链与生态包	39
6.10	LaTeX → Typst 转换	40
6.11	绘图与曲线算法	41
6.12	图片与 PDF 兼容性	41
6.13	编译器机制杂记	41
7	第六部分 群聊精选 (2026-07-01 07-08)	43
7.1	公式 HTML 导出 (MathML / SVG / 性能)	43
7.2	语言特性 / 数据结构	43
7.3	编号 / 计数器	43
7.4	脚注 / 尾注 / 版面	44
7.5	排版 bug / 版本差异	44
7.6	绘图 / 盒模型	45
7.7	字体	45
7.8	文献 / bib	46
7.9	工具链 / 生态包	46
7.10	其它	49
8	第七部分 群聊精选 (2026-07-08 起)	50
8.1	字符 / 字体 / 颜色	50
8.2	文字描边 / 阴影 / 行距	51
8.3	数学	51
8.4	绘图 / 曲线	51
8.5	布局 / 对齐	52
8.6	编号 / 文献 (GB/T 7714 续)	52
8.7	工具链 / 插件	54
8.8	语言陷阱 / AI 协作	54
8.9	导出 / 多端	56

8.10 包 / 资源	56
-------------------	----

2 第一部分 群聊精选（早期，seq <= 269536）

2.1 列表 / 编号

1. 改 **list marker** 颜色不动文字：marker 参数可传 content，外套 text()。#set list(marker: text(fill: red)[1])。图（截图 8f9e63b9 未包含在本地资源中）
2. 常考题：第一层 **indent 1em**、嵌套层 **0em**：#set list(indent: 1em) + #show list: it => { set list(indent: 0pt); it }（set 只影响内部渲染）。图（截图 63a06c72 未包含在本地资源中）
3. 自定义 marker：#set list(marker: box(fill: black, inset: 0.2em)); #set enum(numbering: n => "→") numbering 可为函数。图（截图 a9f90741 未包含在本地资源中）
4. 0.15 better list: #set enum(number-align: horizon)（新参数）；list 用 #set list(marker: align(horizon)[--]); itemize 仍可透传 baseline（PR 8150 已合并）。图（截图 6399c793 未包含在本地资源中）

2.2 标题 / 目录

5. 附录覆盖 **heading numbering**：#set heading(numbering: "1.1") 后，附录用 #set heading(numbering: (_, ..nums) => numbering("1", ..nums))（吞掉第一级参数）。图（截图 92e5e8d1 未包含在本地资源中）
6. 目录文字与正文标题不同：局部作用域 #[#show heading: "章一第" = 第一章]。图（截图 69c41112 未包含在本地资源中）
7. 小节跨章连续编号（**1.3→2.4**）：#show heading.where(level: 2): set heading(numbering: (..) => numbering("1.", query(heading.where(level: 2).before(here())).len()))。图（截图 e7d80ce3 未包含在本地资源中）（另有 state 版：打字机方案，“操作比较危险”）
8. 单独重置三级标题 **counter**：#counter(heading).update(x => (x.at(0), x.at(1), 0)) 思路（context 中取出改第 3 个再 update）。图（截图 b7d31ef3 未包含在本地资源中）
9. 目录条目字体与正文不同：#show outline.entry: set text(font: ...); 进阶：标题尾放 metadata("字体名"), show outline.entry 中 set text(font: x.element.body.children.last().value)。图（截图 4c46c3a0 未包含在本地资源中）
10. 多文件 include 各文档独立 outline：包 suboutline / #set heading(offset: 1)。
11. 目录条目无虚线无页码：给该标题加 label，在 show outline.entry 中按 label 分支。

2.3 段落 / 中文排版

12. 行间公式是否终结段落（first-line-indent all:false 后仍缩进）：#set par(first-line-indent: (amount: 2em, all: false)) 中块级公式后文本仍算新段。FAQ 文档 <https://typst.dev/guide/FAQ/block-equation-in-paragraph.html>。图（截图 e87a88b9 未包含在本地资源中） / （截图 57fb8062 未包含在本地资源中）
13. 智能引号中文单引号误判为 **apostrophe**：CJK 后不空格的 ' 被当英文撇号。FAQ <https://typst-doc-cn.github.io/guide/FAQ/smartquote-font.html>；已知缺陷 issue 8191（gap.zhtyp.art/#smartquote-wrong）。推荐中文直接打全角引号或用「」。图（截图 9457222f 未包含在本地资源中）
14. 连字符断词：text.hyphenate 默认跟 par.justify 一致，且 lang: "zh" 会 suppress。图（截图 423cb786 未包含在本地资源中）
15. 中文换行去空格 FAQ: <https://typst-doc-cn.github.io/guide/FAQ/chinese-remove-space.html>
16. CJK 标点挤压在 text()/show 后消失：issue 5474（字体分段导致）。
17. 中英文省略号同为 U+2026，不区分。

2.4 数学

18. **math.op** 后间距：op 后是 THIN（TeX 规则，process.rs，参考 TeXBook p170）；可用 typst eval -format yaml 查看语法树（op 后跟显式 space 节点）。

19. 行内公式显示成 **display** 后行高异常: `#show math.equation.where(block: false): math.display` 会撑爆行; 修 0.15; 配合 `#show math.equation: set text(top-edge: "bounds", bottom-edge: "bounds")`。图 (截图 90df9884 未包含在本地资源中) / (截图 bfe393e6 未包含在本地资源中)
20. 给等号拉长 (**stretch**): 等号拉不长是字体问题 (STIX2), 换 New Computer Modern Math 解决; 或第三方包。图 (截图 9abdc1a2 未包含在本地资源中)
21. 可变长等号手写: `#let xlongequal(top, bottom: none) = math.attach(math.stretch(math.eq), t: math.limits(top), b: bottom); xarrow` 包与其冲突 (limits 叠加)。
22. 行间公式: 有 **label** 才编号: `#show math.equation.where(block: true): it => { if not it.has("label") { let fields = it.fields(); let _ = fields.remove("body"); fields.numbering = none; [#counter(math.equation).update(v => v - 1)#math.equation(..fields, it.body)] } else { it } }` (重建元素 fields 技巧)。图 (截图 cd49f07f 未包含在本地资源中)
23. 导数竖线: `physica evaluated(dv(f,x))_(x=x_0)` 或 `lr()` 调高度。图 (截图 b586d881 未包含在本地资源中)
24. `$a$` 与 `$ a $` 不同 (后者 block 参数判断为 display-like), box 包裹宽度异常。图 (截图 d6ealee2 未包含在本地资源中)
25. 字符串→公式: `eval` 带 `mode: math` 参数。

2.5 表格

26. “修改某一列的行高”是伪需求——行高整表共享: `rows` 参数 `rows: (0.9cm,) * 9 + (1.6cm,) + ...`。图 (截图 cc05f5d7 未包含在本地资源中)
27. 表格长单词换行: `set text(hyphenate: true) / lang` 注意。
28. `zebrar` 消除 `hline` 干扰: `table.hline(y: ..., stroke: 0pt)`。

2.6 图形 / 页面

29. 并排独立 **figure** + 各自 **label**: `#grid[#figure(image(...)) <a>][#figure(image(...)) <b>]`——`script mode` 中不能 attach label, 必须 [] 回 `markup mode`。图 (截图 8ceaf155 未包含在本地资源中)
30. `i-figured` 包 `ref` 编号不匹配 `caption`: 用 `@<kind>:<label>` 前缀。
31. 图片高度自动填满剩余页面: `grid` 里 `height: 1fr` (需套 `figure/block?` 讨论中 `r` 学到)。图 (截图 0299e884 未包含在本地资源中) / (截图 fbf635de 未包含在本地资源中)
32. `html` 导出不能设置页面大小, 样式自己写 `CSS`; `figure show` 后 `img` 外多余 `p` 标签 `issue 8366` (用 `html.figure` 或 `block(html.img())`)。
33. `PDF` 插入失败: 个例 `PDF` 格式问题, 转格式解决。
34. 横排纸张: `#set page(flipped: true)` (有人拼错 `filpped`)。

2.7 show 规则 / 作用域

35. **layout did not converge / states** 自更新: `pinit` 包问题; `show` 开关用 `state` 更新不收敛 → 自己写函数包一层 / 0.15 用 `within (selector.within)`。revoke `show rule` 仍在 roadmap。
36. `show regex("n+")` 疑与 353 页后 `PDF grouping depth exceeded` 有关 (`max-pdf-grouping-depth` 论坛帖)。
37. `#show heading.where(level:3): it => "(" + it.body + ")"` (`heading` 变形)。
38. `figure` 收集 `image` 路径: `show image` 拿 `it.path` 是相对路径字符串, 最新 `commit` 需 `image(path(...))` 才有 `Path` 对象 (`bundle` 收集 `asset` 场景)。

2.8 字体

39. 字体名填什么: 填 `family name` 而非文件名; `typst fonts` 列本机字体; `Noto Serif CJK SC` 各种候选名逐一尝试; `otfinfo -i / fc-query` 看注册名。
40. 思源黑体 **PNG/SVG** 导出豆腐块: `TrueType` 字体 65536 `glyph bug` (8434), `workaround` 用 `OTF` 版。
41. 中英文分别调字号: `FAQ lang-font-size` (`guide.typst.dev/FAQ/lang-font-size.html`)。

- 42. 可变字体: 0.15 dev 支持 (`text(weight: n*100)` 无级渐变); web app VF 字体优先。图 (截图 e88670c5 未包含在本地资源中)
- 43. 缺字体报错处理: 装字体或改模板定义 (`ctrl+click` 跳转定义处)。

2.9 参考文献

- 44. hayagriva bib 非法 editortype: 转 yaml 时报 `PersonRole::Unknown cannot be serialized` (不报错生成列表)。
- 45. gb-7714-2015-note 样式 ABA 引用显示空白 bug (`typst 6612/clreq108`); omni-gb7714 可接管解决。
- 46. 多 bib: 0.15 原生 multi-bib (一个文档多个 bibliography)。
- 47. citrus 包 CSL 引号→书名号问题。
- 48. bib school 字段: CSL `publisher := bib publisher` 或回落 `institution/school`; PR hayagriva484 等。

2.10 工具链 / 其他

- 49. md→typst: pandoc / cmarker / 大模型。
- 50. zed 无 PDF preview: tinydist 系统浏览器预览; `typst watch onsave` vs 浏览器实时。
- 51. typstyle 0.14.21 删 dots bug (tinydist builtin 版本问题, 0.14.4 正常)。
- 52. 尾注 **endnote**: `counter("endnote") + [#metadata(body)<endnote>] + query(<endnote>).map(x => x.value).map(enum.item).join()`; 改进版 `selector(<endnote>).before(here())` 只显示之前条目 + `<endnote-list>` 标记分段重置。图 (截图 a33e6433 未包含在本地资源中) / (截图 a66d3421 未包含在本地资源中)
- 53. 选择题 **ABCD** 自动平铺: `flex-grid` (`layout + measure` 最大宽 + `floor(ratio)` 限 4 列、`n==3` 时 2 列) + `options()` 解析 `list children` 加 `numbering("A",i)`。图 (截图 fb6c62a5 未包含在本地资源中) / (截图 1a4f0714 未包含在本地资源中)
- 54. dict 值进 table: script mode 无 `int→content` 隐式转换, `.map(str)` 或 `map(it => [#it])`。图 (截图 3efa7700 未包含在本地资源中)
- 55. 函数传参值/引用: 不可变, 全当传值即可 (runtime COW 优化)。
- 56. 0.15 特性省流: `vf`、`bundle export`、`mathml`、`color.spot`、`better list`、`multi bib`、`path`、`typst docs in typst`、`divider`; 小的: `dict/arguments map/filter`、`range.inclusive`、`int.min/max/base`、`list.marker-align`、`selector.within`、`counter.display(at:)`、`typst eval` 取代 `typst query`、`pdf/svg` 压缩。
- 57. bundle 导出共享页码: 8389 有解 (图 (截图 943f74f4 未包含在本地资源中) ...))。
- 58. mathml 定位是无障碍, 不是替代 SVG 渲染; 浏览器字体差异 (chromium 无默认数学字体、firefox 带 Fira Math?)。
- 59. 群机器人 FAQ 指令: `/typ /tytyp /typdev /typdev eval /typm`, 页面设置说明 (横向 A8 等)。
- 60. 竖排: `vtzone` 包 (简单场景, 不处理中文标点); `box width 1em` 土法; 0.15 无原生。
- 61. 连字取消: `#set text(features: (calt: 0))` (Cascadia Code 等编程字体用 `calt` 实现连字)。图 (截图 400653eb 未包含在本地资源中) / (截图 174df3bd 未包含在本地资源中)
- 62. zebra 用法: `line-range: (1, 9)` 取行、`highlight-lines`、`caption` 拼路径变量。
- 63. 图+文字绕排包: `meander / warp-it`。
- 64. obsidian typst mate 中文方块: 字体路径问题 issue46。
- 65. 高考试卷排版热: `ezexam` 包 (密封线); Charlie 排 2026 新高考 I/II 卷。

3 第二部分 群聊精选 (2026-06-17 06-24)

3.1 行距与基线 (排版原理类)

66. **leading** 的真实语义: `set par(leading)` 是“上一行 bottom-edge 到下一行 top-edge”的距离, 不是基线间距。文字盒上下边缘由字体 metrics 决定, 中西文混排时同一行盒高度不同 → 行距看起来被“挤压”。想直接控制基线间距: `#set text(top-edge: "baseline")`, 此后 **leading** 即基线距。(问答: 中文行距忽大忽小)
67. 稿纸/格线效果配方 (seq 268591): 让每行盒高度固定为 **step**, 行与行之间也留 **step**, 即可与背景 **tiling** 横线严格对齐:

```
#set text(top-edge: 0pt, bottom-edge: 0pt)
#set par(leading: step, spacing: step * 2, justify: true)
```

配合 **paint/tiling** 横线背景即得“作文格/稿纸”。若只是想让中西文混排行距一致, 简单方案 `#set text(top-edge: 1em)`。

68. **tiling** 渲染一致性坑: `tiling(pattern)` 背景格线与 **grid stroke** 在 **tinymist web** 预览中高度对不齐, 但本地 PDF/PNG 导出一致; 同一 PDF 在不同阅读器 (Chrome/Edge/Sumatra) 下 **pattern** 填充行为也不同。结论: 需要精确对齐的场景不要用 **tiling**, 改用 `place(..., grid(...))` 手动画线。导 PNG 分别用 PPI 280/290/300 可复现差异。(图: (该效果截图未包含在本地资源中) 未落盘, 文字描述即可)
69. 可变字体 (VF) 导出回归 (0.15 已知坑): 思源黑体 Source Han Sans SC 的 VF ttf/ttc 会导致 PNG/SVG 导出字形错乱 (PDF 正常); Noto Sans SC 正常。临时方案: 删 VF 用 **static otf**。根因在 **ttf-parser/harfbuzz** 对 VF 的处理; 修复方向: 移除 **ttf-parser** 换 **skrifa** (typst.ts PR858); **tinymist issue2585**。

3.2 盒模型 / 布局技巧

70. “邪修”嵌套盒左列填充 (图 (截图 f5d99d8f 未包含在本地资源中) + (截图 8797fa26 未包含在本地资源中)): 想让 **box** 内黄色左块撑满整行高度, 用超大 **outset** 把视觉盒“推出去”再靠外层 **clip** 裁回:

```
#let f(l, r) = {
  let inset = 5pt
  box(clip: true, radius: 5pt, inset: (y: inset, right: inset), stroke: 1pt, {
    box(l, fill: yellow, inset: (x: inset), outset: (y: 999em))
  })
  r
}
```

原理: **outset: (y: 999em)** 让黄色盒的绘制范围纵向超出父盒, 父盒 **clip: true** 裁剪 → 视觉上左列填满。

71. 小盒对齐基线 (0.15 pr8150): **math box** 现在带 **baseline** 参数, **fletcher** 流程图的分数线可与节点基线对齐。实现: 零宽 **box** 两次 **measure** 求偏移。

3.3 数学排版专题

72. **lrcases** 手写方案 (图 (截图 7363bf29 未包含在本地资源中)): 左花括号右对齐的 **cases**:

```
#let lrcases(it, size: 100%) =
  math.lr({$ + box(baseline: (at: horizon, shift: -0.25em), it, ), size: size}$)
```

baseline: (at: horizon) 让括号垂直中心对准数学轴; **shift: -0.25em** 的来源: NCMM 等 CM 系字体 OpenType MathConstants 的 **axisHeight** $\approx 0.25em$ (issue8516, 测量样例 **typst.app/project/rkMI42sAZEj53EToDpy5zY**)。

73. **fake-cases** (**fa_555** 版) (图 (截图 fd2b7252 未包含在本地资源中)):


```
#let fake-cases(gap: .6em, eq) = {
  set par(leading: gap)
  math.lr(sym.brace + block(inset: (y: gap / 2), eq))
}
```

用 brace + block 拼 cases，行距由 par leading 控制。

74. **sym.angle** 全变体表（图（截图 a28bceaf 未包含在本地资源中） / （截图 cd38bc3b 未包含在本地资源中））：`#repr(sym.angle)` 打印出 17 个变体：`sym.angle(∠)`、`.arc(—)`、`.right(L)`、`.right.dot(⊥)`、`.spheric(◁?)`、`.rev(⊥)` 等——需要“夹角/球面角/直角标记”符号时先 repr 查表。
75. 多字母数学量应进文本模式：`$amount$` 会被逐字母斜体成 `a·m·o·u·n·t` 且 kerning 怪异，这是设计使然而非 bug。正确：`$ "amount" $` 或 `italic("amount")`。
76. 重音吃掉 i/j 的点是故意的(issue7243)：`hat(i)` 无点。想要 hat+点：`hat(s)_j` 手法或 `dotless i` 设 false。
77. **cases** 逐行编号：用 **equate** 包（社区方案），原生 numbering 不支持行级。
78. **MathML** 导出小瑕疵：`<mo>` 缺 `stretchy="false"` 导致运算符间距怪异（issue 已记录）。
79. **OpenType** 特性调数学字形：`#text(features: (ssty: 2,), $x$)` 切换 superscript 字形（`ssty=1/2`）；`cv01` 等是斜线零/字符变体：`#show math.equation: set text(features: ("cv01",))`。
80. **0.15** 内置 **NCM** 字体升级改 ∅ 形状（7597→7.1.0、pr8435→8.1.0）：`\emptyset` 默认从斜穿变圆零。恢复斜穿：`$emptyset.zero$` 或 `set text(features: ("cv02",))`。（图（截图 94b999d1 未包含在本地资源中））
81. **typst** 二进制内置 **Libertinus + New Computer Modern**（约 19.7MB），无需装字体即可用 `$calcs$` 等。

3.4 语言 / 数据结构讨论

82. 数组、字典都能解构，**pair** 不能：`#let (x, y) = (1, 2)`；`dict` 也行：`let (width, height) = measure[114514]`（按 key 顺序取值是 array 语义——实际是对 `dict.items` 顺序解构）。`pair` 只能 `.x/.y` 或 `.0/.1`。
83. 参数默认值合并惯用法（图（截图 2be86420 未包含在本地资源中））：`spread` 合并：`let default = (left: lem, ...); (:...default, ..input)`；或 `dict+ 运算符`（浅合并）。递归合并官方没有，需自己写函数。
84. 语言设计闲谈：`positional` 参数 = `named` 参数（`f(1)` 即 `f(x: 1)`）的历史原因；`a: b` 与 `a = b` 冲突导致 `dict` 字面量语法演变；`match`/穷尽检查在动态类型语言里容易退化成 `any` 所以难做；`wasm` 插件限制：无网络 API、单线程。
85. **0.15** 新选择器 **selector.within**：`figure.where(kind: image).within(heading.where(level: 1))`——注意 `revoke style`（样式撤销）仍未实现。

3.5 编号 / 文档结构

86. 代码块按章编号：代码块即 `figure.where(kind: raw)`。混合编号：

```
#show figure.where(kind: image).or(figure.where(kind: table)).or(figure.where(kind:
raw)):
  set figure(numbering: num => numbering("I.1", counter(heading).get().first(),
num))
```

（FAQ 图（截图 e6239125 未包含在本地资源中））

87. 每章从 1 重新计数：章标题触发时重置各 counter：

```
#show heading.where(level: 1): it => {
  counter(figure.where(kind: image)).update(0)
  // raw / table 同理
  it
}
```

连续章节号片段可参考 `ouc-bachelor-thesis` 模板 `utils/chapnum.typ`。

88. 去页眉按页判断 (seq 269467 269476): 非固定页面 (如目录/首页) 想去页眉: 在 `header` 里 `context` 取 `counter(page).get().first()` 判断, 或 `query metadata` 与 `here()` 位置比较 (`metadata` 打点标记“此页无页眉”)。

3.6 FAQ 机器人 (/typ 指令完整说明, part_004 帮助文本)

89. 用法汇总: `/typtyp` (文档) (typst 语法高亮)、`/typ` (文档) (按内容自动伸缩页面)、`/typm` (数学)、`/typ eval` (表达式) (直接求值)、`/typdev` (dev 版编译)、`/typtyp fonts` (列字体)。默认中文字体 Noto Serif CJK SC。引用机器人消息: 被引用的消息内容存为 `re.typ` 可 `#import "re.typ": */include / read`。行首 `!!{package}` 自动展开 `#import "@preview/{package}:最新版"`。误发代码可撤回, 跟随撤回渲染结果。→ 这是群里“代码-效果图”对偶的来源。

3.7 工具链 / 生态

- 90. `typst watch` 自动重编译; 调试: 官方无 debugger, 社区用 `panic()` 顶替打印中间值 + 实时预览观察。
- 91. **tinymist**: `pinMain` 配置 (多个 `include` 候选时报错让选); `preview` 点击跳源码 (PR2055); 补全后 `delete` 行为 issue2514; `webview` 渲染大文档卡顿 (SVG vs canvas 之争、DOM tree 压力; 拖成独立窗口可缓解); `lockfile` 依赖锁定; `shiroa` 新版支持 `--input`。
- 92. `datetime.today().display()` 默认截到“日”是为编译确定性/缓存 (issue1988)——同一文档全天输出相同; 要精确时间用 `datetime.now().display("[hour]:[minute]")` 需关缓存或接受每次变。
- 93. **WASM** 内跑 **LLM** 实验: `wasmtime fork` 加 3D 渲染 +13% 体积, `cetz` 场景负提升——结论: `typst` 插件沙箱跑大模型暂无性价比。
- 94. 生态新包: **conjak** (农历/干支/节气日期包); **typsium 0.3.2** (化学式, `#ce` 新语法支持 $^{(2+)}$ 或 content 写法); `merman` (mermaid); `column-major grid` (转置表格, `files` 目录有 `column-major-table.typ`); `velyst` (bevy 动画嵌入); `InkyCap` (tauri + obsidian 式编辑器); `text-shadow` 方案 (guide FAQ + SVG `feDropShadow`: `typst.app/project/rloneBy3UYs3pJlkO8t5t3`); `heading` 悬挂缩进多行对齐 (`gap.zhtyp.art/#heading-hanging-indent + issue6527`)。
- 95. 解构效果验证图: (该效果截图未包含在本地资源中) (`dict` 解构 `let (width, height) = measure[...]` 的 FAQ 渲染截图)。

3.8 可嵌入资源 (本轮核实存在)

- 全部图片路径已 `grep` 核实于 `E:groupresourcesimages` (311 个文件), 上列 hash 均匹配。
- `resourcesfiles`

值得检查: `3de2345755ed8e1903f8a75fc5f52d43_column-major-table.typ`、`4bfff643b117334f202a37b69f4501bdb_讲义模板.typ`、`a8a110517f3d513d162130fe8b6b33aa_tennis-parabola.typ`、`3790ca520ca0827e8bcc598eb6a9feb6_curve.typ`。

4 第三部分 群聊精选 (2026-06-24 06-27)

4.1 数学排版

96. 多字母数学量必须用引号: Typst 数学模式下连续字母会被当作多个单字母变量相乘 (`amount = a m o u n t`), 要写成一个量需加引号 `$ amount $`。群里讨论认为这符合写代码的人的直觉, 且“归根结底多字母数学量不是好写法”(单字母才是规范写法)。
97. 定界符左右大小不一致 → 用 `lr(size:)` 手动指定: 截图里左右两个 `|` 框看上去不一样大, 原因是 `| a / b |` 两侧内容高度不同 (如 `a` 比 `b` 矮), 自动计算高度就不一样。官方做法是用 `lr` 的 `size:` 参数手动调整取整符号大小; 另有人提议“在 `a` 旁边放一个隐藏的 `b`”来撑高。字体为 `libertinus math`。

```
$
  lr(|, size: #1em)a / b | = abs(b / b, size: #1em)
$
```

$$\left|\frac{a}{b}\right| = \left|\frac{b}{b}\right|$$

▲ 代码实测 (本机 typst 渲染)

98. 中文括号里塞数学公式会很难看: 直接写 (`$\lr{bar.v S bar.v}$`) 时中文括号与公式字体不搭。讨论结论是主要问题在定界符——大部分中文字体的英文字形做过特殊调整, 而数学公式字体没有这种调整, 两者放一起非常突兀。绕过办法: ① 改用英文括号 (`$(\lr{bar.v S bar.v})$`); ② 用 `show` 把中文括号替换成英文/智能引号 (有人用同样的 `show` 大法解决了中文 `"` 错位问题); ③ 用 LaTeX 同款字体。也有人指出 `replace` 在编辑器层面也能做, 但对外部数据无效。

```
$(\lr{bar.v S bar.v})$
$(\lr{bar.v S bar.v})$
```

```
(|S|)
(|S|)
```

▲ 代码实测 (本机 typst 渲染)

99. 数学字体风格选型讨论: Times 风格本身没有良好定义 (Times 经过多轮修改, 衍生改进的都算); Word 公式用的是 **Cambria Math**, 普通字符用 Word 英文/中文字体、数学与特殊字符用 Segoe UI Symbol; 其他备选有 **STIX Two**、**texgyretermesmath**。吐槽: `libertinus math` “到处透露出半成品”的样子, New Computer Modern 太肥。
100. 应对“数学字体必须是 Times New Roman”的查重系统: 有学校本科毕设格式检测要求数学字体为 TNR (原理: 非汉字字符都得是 TNR, 计算比例超过阈值; 疑似硬编码字体名 “times new roman”)。LaTeX 里可用 `unicode-math` 的 `range` 把数学环境映射到 TNR 字符凑阈值 (但 `range` bug 频发, 且缺码点如 Sigma、拉伸括号)。Typst 侧的探索:
- 导入 **natrrix** 包 <https://typst.app/universe/package/natrrix>, 自己把 `delim` 传入需要的参数;
 - 用 **font covers regex** 实现类似 `range` 的效果 (“比 `range` 灵活不少”, 但 “用来干这个非常灵车”);
 - 想通关要么把公式改成图片, 要么改成 TNR 伪斜体;
 - 还有人提议 fork 一个字体、把名字改成 TNR 那套信息骗过系统。
101. **Windows** 下数学 **variant style** 全部失效: Windows 上所有数学字体的 `serif/sans/frak/scr` 等 `variant` 都不能用, Linux 下正常 (用默认字体 New Computer Modern Math)。猜测原因是 Windows 环境另外装了残缺版 New Computer Modern Math, Typst 优先使用了系统装的而非内置字体; 后来“复现不了了”, 未确认是 typst CLI 还是 `tinymist` 的现象。

```
#show math.equation: set text(font: "New Computer Modern Math") // 中间有 Modern
$ serif(Z z Zeta zeta) & thick & sans(Z z Zeta zeta) \
```

```
mono(Z z Zeta zeta) & & bb(Z z Zeta zeta) \
frac(Z z Zeta zeta) & & scr(Z z Zeta zeta) $
```

$ZzZ\zeta \quad ZzZ\zeta$
 $ZzZ\zeta \quad \mathbb{Z}zZ\zeta$
 $\mathfrak{Z}zZ\zeta \quad \mathcal{Z}zZ\zeta$

▲ 代码实测（本机 typst 渲染）

102. 文本模式的上下标叠放（**sub + sup** 同时出现在右上角）：除数学模式的 $_{}^{\wedge}$ 外，文本模式没有原生支持，“文本模式的相关功能还没做”，只能 workaround；斜体等边缘情况照顾起来麻烦。群里迭代了多种写法：

- 用 h 负偏移手工对齐（有人从 /typd 跑出来验证）：

```
x#sub("a")#context{h(-measure(sub("a")).width)}#super("b")
```

x_a^b

▲ 代码实测（本机 typst 渲染）

- place 版（LLM vibe 出来的，但尺寸没搞对，烧完了配额）：

```
A#place[#sub[sub]]#super[sup]
A#box(place[#sub[sub]])#super[sup]
```

A
 $\begin{matrix} \text{sup} \\ \text{sub} \end{matrix} A \begin{matrix} \text{sup} \\ \text{sub} \end{matrix}$

▲ 代码实测（本机 typst 渲染）

- box + stack 版：

```
#let fun(a,b)=box(height: 0.8em, text(size: .6em, stack(dir: ttb, spacing: .4em, a, b)))
A#fun[sup][sub]
```

$A_{\text{sub}}^{\text{sup}}$

▲ 代码实测（本机 typst 渲染）

- 最终较优的 grid 版（配图评论“好神秘”）：

```
#let sub-sup(sub, sup) = {
  set text(size: 0.5em)
  box(
    baseline: 0.2em,
    grid(
      rows: 0.8em,
      grid.cell(align: top, sup),
      grid.cell(align: bottom, sub)
    )
  )
}
a#sub-sup([2], [1]) a#super[1] a#sub[2]
```

$a_2^1 a_1^1 a_2$

▲ 代码实测（本机 typst 渲染）

另有人指出部分字符本身有现成 `code point` 可用。需求截图见。

103. 数学公式里 **tuple** 冲突的临时处理：写模型论公式时先 `#let tuple = none`；把名字占掉，避免 `tuple(...)` 被解析成内置函数。原始提问“何意味对齐”指向 \ 换行对齐的表现。

```
#let tuple = none;

$
x in A->((tuple(A, R) models psi(a_1, dots.c, a_n, x)) & <-> (B models psi(F(a_1),
dots.c, F(a_n), F(x)))) \
x in A->(tuple(A, R) models psi(a_1, dots.c, a_n, x)) & <-> x in A->(B models
psi(F(a_1), dots.c, F(a_n), F(x))) \
$
```

$$x \in A \rightarrow (((A, R) \models \psi(a_1, \dots, a_n, x)) \leftrightarrow (B \models \psi(F(a_1), \dots, F(a_n), F(x))))$$
$$x \in A \rightarrow ((A, R) \models \psi(a_1, \dots, a_n, x)) \leftrightarrow x \in A \rightarrow (B \models \psi(F(a_1), \dots, F(a_n), F(x)))$$

▲ 代码实测（本机 `typst` 渲染）

104. **MathML** 输出不支持 `style="color"`：截图显示 **MathML** 实际上支持 `style="color"` 属性，但 **Typst** 当前的 **MathML** 输出还没支持。
105. 数学作图库选型：勾股定理这类证明图可以用 **ctz** 画完再以 **image** 塞进 **Typst**；处理欧式几何（对标 **tkz-euclide**）推荐 **ctz-euclide** <https://typst.app/universe/package/ctz-euclide>；画双曲线则连 **tkz-euclide** 都不方便，可考虑 **asymptote**。另有被评价“这个库画图真好用啊”的 **q.uiver.app** 已支持导出 **Typst**（画交换图）。
106. 流程图/框图类图形：有人问这种图推荐什么包或工具，回答是 **draw.io** 画了再插入。
107. 行内公式高度/深度与自动行距：有人问 **Typst** 文档有没有记录过 `inline equation` 在高度/深度超过多少时开始自动调整行距——群里未给出结论（待查）。

4.2 字体与文字样式

108. 能否在 **Typst** 内部传入“改过名字的字”： `font` 字段能传路径，于是有人想 `read 字体` → `修改字体信息` → `传进去骗过检测系统`。结论：“能传路径的地方都能直接传 `byte`”，但不能在 **Typst** 内部指定字体文件路径——因为一个字体文件也可能包含多个字体，字体搜索跟文件不是一个概念。花活失败。（旁及相关项目：动态生成 `plugin` 的 `C compiler` 封装 <https://github.com/ParaN3xus/typst-xcc-wrapper>）
109. **SimSun** 渲染全变方框是 **0.14** 的 **bug**，**0.15** 已修：本地用 **SimSun** 时中文全部变成方框（英文字体同样情况却没问题，一度被怀疑“**simsum** 全责”或字体本身问题）。确认是 **0.14** 的缺陷，更新到 **0.15** 问题消失。应急：换成 **FangSong**。（**VSCode** 本地效果，**typst.app** 效果）
110. **tinymist preview** 中文渲染不出来，但导出 **PDF** 正常：现象见。涉及的是一个实验报告模板 `lib.typ`（用到 `@preview/itemize:0.2.0`、`@preview/zebraw:0.6.1`、`@preview/gentle-clues:1.3.1`、`@preview/theorion:0.6.0` 与 **cosmos** 主题），完整代码见第 139 条。代码本身编译无问题、生成的 **PDF** 是正常的，只是 **preview** 没中文——根因是字体（见下条）。
111. 上条的根因：**harfbuzz** 对思源 **TTF** 的 **65536** 字形溢出（重要坑）：Source Han Serif 的 **TTF**（**.ttf.ttc**）包含 `u16::MAX + 1 = 65536` 个字形，**harfbuzz** 读取时 `let actual_total = u16::try_from(actual_total).ok()`？溢出导致读取失败 → 见 <https://github.com/typst/typst/issues/8434>。其它 **TTF** 字形没那么多就碰不上。绕过办法：
- 从 <https://help.mirrors.cernet.edu.cn/adobe-fonts/#source-han-serif-download> 重装静态版本或 **OTF VF**；清华镜像 <https://mirrors.tuna.tsinghua.edu.cn/adobe-fonts/source-han-serif/Variable/OTC/> 同样能下到 `otf.ttc`（镜像站镜像的是 `release` 分支，所以也能下到 `ttf.ttc`）；
 - 直接把字体目录下的 `ttf` 文件弄没，让 `otf` 优先（“有可能 `ttf` 碰巧优先于 `otf` 使用了”）；

- 字体名用 Source Han Serif SC 而非 Source Han Serif (有人说这样也行, 也有人回“也不行”, 存在分歧);
- 或干脆在 .typ 里写中文字体名「思源宋体」。

另注: tinymist v0.15.2 用本地装的 source han serif 没问题, 有人怀疑与 win10 + VSCode + tinymist 组合有关。

112. **tinymist** 字体面板怎么看: ctrl+shift+p 搜索 font, 找到 tinymist 相关的那条回车执行即可打开面板。若面板内容显示不全 (“第三行字没显示出来”), 关掉重新打开或重新编译即可。
113. **CJK** 与拉丁之间手动换行产生的多余空格 (版本确认存疑): 有截图显示文本前莫名多出空格, 怀疑是 text 前多了个空格导致。相关讨论指向 0.13.1 能复现, 而 0.13.1 → 0.14.0 修复了一个“有关汉字周边间距调整、且只在非左对齐时能触发”的缺陷; 但后来“回到 14.4 复现不了了”, 可能记错版本号。参考资料 <https://gap.zhtyp.art/#cjk-latin-manual-linebreak> (该站用途之一就是搜 issue)。

```
#set align(center)

#{
  text("我是一句话, \n我是二句话, ")
}
```

我是一句话,
我是二句话,

▲ 代码实测 (本机 typst 渲染)

114. 伪加粗 **cuti** 的 **stroke** 取值: 用 cuti 做假的中文加粗时, stroke: 0.01em 是对比 Word 的黑体加粗得来的值; 有人感觉 cuti 的粗体比 Word 粗一点。可用 PDF 编辑器查看实际 stroke 值核对。
115. 文字阴影的两种实现:
 - **SVG feDropShadow** 法 (来自 <https://www.github.com/typst-doc-cn/guide/pull/170>, 预览 <https://deploy-preview-170--luxury-mochi-9269a9.netlify.app/FAQ/text-shadow.html>)。要点: 布局盒用原生 Typst 文本度量, 只有阴影走 SVG; baseline 度量方式是 ascent = text.top-edge -> "baseline"、descent = "baseline" -> text.bottom-edge。只支持 str body、单一字体、数字 weight。原消息代码被截断, 以下为可见部分:

```
#let xml-escape(s) = { s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;"); } // 粗略实现 xml.encode#let xml-elem(tag, attrs: (:), ..children) =
{ assert(children.named() == (:)) and children.pos().len() <= 1) "<" + tag for (k, v)
in attrs { " {k}='{v}'" .replace("{k}", k) .replace("{v}", if type(v) == length
{ str(v.pt()) } else if type(v) == color { v.to-hex() } else { str(v) }) } ">"
children.pos().first(default: none) "</" + tag + ">"} // SVG drop-shadow text, but
its layout box is measured from native Typst text.#let drop-shadow( // 只支持 str
body, // 以下是阴影参数, 详细含义可参考 SVG <feDropShadow> // https://developer.mozilla.
org/en-US/docs/Web/SVG/Reference/Element/feDropShadow shadow-dx: 12%, shadow-dy:
12%, shadow-std-deviation: 6%, shadow-color: black, shadow-opacity: 0.35, // 以下参数
同 typst 的 text 元素, 但要改成 SVG 写法 // 只测试过单一字体 font: "Noto Serif", // 只支持数
字 weight: 700,) = context { let body-text = text.with(font: font, body) let layout-
w = measure(body-text()).width // baseline metrics: // ascent = text.top-edge ->
"baseline" // descent = "baseline" -> text.bottom-edge let ascent = measure(body-
text(bottom-edge: "baseline")).height let descent = measure(body-text(top-edge:
"baseline")).height let layout-h = ascent + descent let dx = shadow-dx * text.size
let dy = shadow-dy * text.size let blur = shadow-std-deviation * text.size let pad-
left = 3 * blur let pad-right = dx + 3 * blur let pad-top = 3 * blur let pad-bottom
= dy + 3 * blur let svg-w-len = layout-w + pad-left + pad-right let svg-h-len =
layout-h + pad-top + pad-bottom let svg = xml-elem( "svg", attrs: ( xmlns: " http://
```



```
www.w3.org/2000/svg ", viewBox: (0pt, 0pt, svg-w-len, svg-h-len).map(x =>
str(x.pt())).join(" "), ), { ... (原消息在此处被截断) ... }
```

- 纯 **Typst** 多层 **place** 叠加法（简单但占内存）。注：原代码里 **dy** 一行误写成 **delta * i**，应为 *** j**：

```
#let shadow(data, dx: .075em, dy: .1em, delta: .1pt) = {
  set par(
    first-line-indent: 0em
  )
  box({
    for i in range(-2, 3) {
      for j in range(-2, 3) {
        place(
          dx: dx + delta * i,
          dy: dy + delta * i,
          text(
            fill: rgb("C0C0C010"),
            data
          )
        )
      }
    }
    data
  })
}
```

#shadow()[Typst]

Typst

▲ 代码实测（本机 typst 渲染）

另有人问 SVG 里的 `<rect>` 有什么用——似乎删掉了也没问题。

4.3 盒模型与布局

116. **box** 的 **width/height** 与 **clip** 尺寸不一致（疑似版本行为变化）：用外层 **box** 装照片、顶部对齐再 **clip** 裁掉多余高度时，画出 **box** 边框是正方形，但 **measure** 量出来的却是没裁切过的尺寸——外层 **box** 的计算尺寸似乎跟内层取了个 **max**。旧版（0.14 时）正常，疑似顺手修的 **bug**： **clip** 说的是 **hidden** 而非 **removed**。绕过办法：给外层 **box** 显式设置 **width**、**height** 就行。对比写法：

```
#box(width: 100pt, height: 100pt, clip: true, { set align(center + horizon)
scale(100% / 1.5, body, reflow: true)})
```

117. 任意缩放内容到指定尺寸（**rescale**，**cover** 模式）：用于“随意改变一个字的长宽比”等需求，官方文档参考 <https://typst.app/docs/reference/layout/scale/>。原代码来自 https://github.com/vanleefxp/tessera_typ/blob/e583c5b24d6aa1bf81ccdbbba6278d8fba0cac71/src/_impl/transform.typ#L46-L97（群友自述“我从注释那个 GitHub 链接里复制的”，另一人回“那应该就是我之前写的”）。

```
#set page(height: auto, width: auto, margin: 1em)// https://github.com/vanleefxp/
tessera_typ/blob/e583c5b24d6aa1bf81ccdbbba6278d8fba0cac71/src/_impl/transform.typ#L
46-L97 #let rescale( body, width: auto, height: auto, fit: "cover", alignment:
center + horizon,) = { if height == auto { panic("omitted") } else if width == auto
{ panic("omitted") } else { let (width: orig-width, height: orig-height) =
measure(body) if fit == "stretch" { panic("omitted") } else { let width-scale-ratio
```

```
= width / orig-width * 100% let h1 = width-scale-ratio * orig-height let scale-ratio
= if h1 < height { height / orig-height * 100% } else { width-scale-ratio }
box(clip: true, align(alignment, scale( box(body, width: orig-width, height: orig-
height), scale-ratio, reflow: true, ))) } }#let body = block(width: 200pt, height:
150pt, fill: orange, { set line(stroke: green + 10pt) place(line(start: (0%, 0%),
end: (100%, 100%))) place(line(start: (0%, 100%), end: (100%, 0%)))
place(line(start: (25%, 0%), end: (25%, 100%))) place(line(start: (0%, 75%), end:
(100%, 75%)))})Lorem#context rescale(body, width: 100pt, height: 100pt)Lorem
```

118. 行距 **leading** 对数学公式盒高的诡异影响：把 `text` 的 `top-edge`/`bottom-edge` 都设为 `"baseline"`、`par.leading` 分别设为 `0pt` / `6pt` / `12pt` 时，同一个内层 `box(height: 0pt)` 包 `$V$` 与 `$s=\mat(1,I;I,1)$` 的表现不同。有人说 `align(bottom, ...)` 能解决但不知为什么；还有人抱怨“`typst` 是不是甚至没有办法测量一个 `box` 的基线到 `bounds` 的距离”→答：“去拆 `context` 和 `content` 就行了”。疑似 `Typst` bug（待确认）。

```
#set text(
  size: 12pt,
  top-edge: "baseline",
  bottom-edge: "baseline",
)

#set par(leading: 0pt)

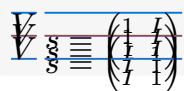
#box(stroke: 0.7pt+red, box(height:0pt, stroke: 0.5pt+blue, $V$))
#box(stroke: 0.7pt+red, box(height:0pt, stroke: 0.5pt+blue, $s=\mat(1,I;I,1)$))

#set par(leading: 6pt)

#box(stroke: 0.7pt+red, box(height:0pt, stroke: 0.5pt+blue, $V$))
#box(stroke: 0.7pt+red, box(height:0pt, stroke: 0.5pt+blue, $s=\mat(1,I;I,1)$))

#set par(leading: 12pt)

#box(stroke: 0.7pt+red, box(height:0pt, stroke: 0.5pt+blue, $V$))
#box(stroke: 0.7pt+red, box(height:0pt, stroke: 0.5pt+blue, $s=\mat(1,I;I,1)$))
```



▲ 代码实测（本机 `typst` 渲染）

119. **measure** 对数学公式不可靠 / **ascender + descender ≠ height**：实测 `asc + desc` 不等于 `height`，且对数学公式“好像根本就不 `work`”（`desc` 和 `asc` 都等于 `5pt`）。旁人评价“像是 `bug`，可以提 `issue`”；提问者后来“找到了另一个方法”。
120. **#set align(center)** 与 **align(center, ...)** 是等价的：官方论坛明确答复 “Both variants are intended to be equivalent and unless you are manually parsing the content or something like that, there shouldn’t be any difference.” → <https://forum.typst.app/t/is-there-any-difference-between-set-align-center-and-align-center/5666>
121. 设置 **set** 的作用域：`set` 本身没有作用域，但它外面的 `{...}` 有。（在排查页眉被污染问题时提出）

4.4 表格

122. 列优先（**column-major**）表格需求：默认 `table` 是从左往右、从上往下填充；有人想从上往下、从左往右填充，用来做细长表格（如 `x 1 2 3 4 5 6 7 / f(x) 1 2 3 4 5 6 7` 这种自变量-函数值对应表）。三种方案：

- 先转置数据再扔进 **table**: 但这样就用不了 **table.header** 了, 顶多用 **table.header** 的手工替代 (**header** 不可替代的用途主要是换页时重复表头);
- 整体旋转 + 单元格内容反向旋转 (“请两次旋转出山”): 注意 **rotate** 的布局范围不会跟着旋转, 长宽为 **auto** 时比较可行; 但 **table.header** 在 **paged export** 时会被忽略 (`// header was ignored during paged export`):

```
#set page(width: 60em, height: auto, margin: 2.5em)

#let horizontal-table(..args) = {
  let contents = args.pos()
  rotate(origin: top + left, reflow: true, -90deg, table(
    ..args.named(), ..contents.map(
      cell => {
        rotate(90deg, cell, reflow: true)
      },
    )
  ))
}

#let contents = {
  range(19000, step: 500).map(
    it => str(it),
  )
}

#table(
  columns: 2,
  // table.header([这是第一行], [第二行]),
  ..contents,
)

#horizontal-table(
  columns: 2,
  // table.header([这是第一行], [第二行]), // header was ignored during paged export
  ..contents,
)
```

- 手动维护矩阵模拟: 群友用 250 行写了一个 **column-major-table** 函数 (`[file:column-major-table.typ]`), 实现方式是手动维护一个矩阵; 效果见 , 另一个表格效果 (作者自评“第三行的实现有点丑陋”)。已知缺口: 忘记处理 **span**, 还没处理 **table.hline** / **table.vline** / **table.cell**。也有人主张这种表格手写就行, 或参考 **eggs** 包里写“莱比锡注释”的换行表格实现; 列优先表格的思维模型是“一个个 **group** 放在一起”。
123. **table.header** 到底有什么用 / 为什么现在不能 **show table.header**: 讨论结论——如果表格不跨页、目标读者不用屏幕阅读器 (比如准备印在纸上、不分发 PDF)、也不导出 **html**, 那 **table.header** 确实没什么用; 若要导出 **html**, **table.header** 里的东西会从 `<td>` 变成 `<th>`, 默认样式不同, 使用它就十分必要; 等未来支持 **show table.header** 后, 导出任意格式都能利用它批量设置样式, 那时估计就没人再怀疑它了。有人吐槽“Typst 文档太强调 AT 和语义了, 想要实现什么都先要考虑语义”。
 124. 字典展开时后出现的键覆盖先出现的: 把 `..args.named()` 展开在函数参数里, 若其中含 **columns**, 则“手动指定的 **columns** 和 **args** 里的 **columns** 哪个在后面哪个生效”。官方依据 (**Dictionaries** 文

档): “✓They can also be spread into a function call or another dictionary with the `..spread` operator. In each case, if a key appears multiple times, the last value will override the others.” <https://typst.app/docs/reference/foundations/dictionary/>

```
#let f(..args) = {  
  table(  
    columns: 7,  
    ..args.named(),  
    ..range(50).map(  
      it => str(it),  
    )  
  )  
}
```

```
#f()
```

```
#f(columns: 10)
```

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	32	33	34
35	36	37	38	39	40	41
42	43	44	45	46	47	48
49						

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49

▲ 代码实测 (本机 typst 渲染)

125. 符号表 / 三线表的手工画法: 用 `table.hline(y: ...)` 配不同 `stroke` 画三线表, 并用 `align: (horizon,) * 2` 让符号列与定义列垂直居中。完整代码见第 139 条末尾注释掉的 `#table(...)` 块 (结构为 `columns: 2`、`rows: (auto, 2em)`、`stroke: none` + 三条 `table.hline(y: 0/1/7)`)。

4.5 语言特性

126. 把 **100%** 这样的 **ratio** 变成字符串 **"100%"**: `str()` 不能直接作用于 `ratio` (“ratio 好像不能用 `str` 函数”), 原来的唯一办法是 `str(repr(100%))`。群友给出两种更干净的做法:

```
{  
  let a = 100% * 1pt  
  str(a.pt() * 100)  
}
```

100

▲ 代码实测 (本机 typst 渲染)

```
#{str(100% / 1%)+"%"}
100%
```

▲ 代码实测（本机 typst 渲染）

- 127. 包内相对路径被解析成包的绝对路径：在包里定义了一个“引入路径”的函数，直接跑文件时不报错，但从包引入时路径变成了“包的绝对路径 + 我写的 path”。群里未给出原因（待查）。
- 128. 调试深嵌套包内变量的土办法：当想看深嵌套包里某个函数里的变量值时，做一个无法 **attach** 到任何东西的 **label** 就可以把变量打印出来，配合 **error lens** 食用更佳；缺点是只能看到第一次的值。有群友问“既然如此为什么不 hover”——回复是在 **package** 里的文件（不是当前 **preview** 的 .typ）hover 不到。
- 129. **error vs panic**——测试覆盖率的价值：有人发现自己的包几个月来一直在使用一个名为 **error** 的不存在的函数（正确写法是 **panic**），因为那行一直没被执行到，直到看测试覆盖率才发现。
- 130. **html.hr** 的内置对应物是 **#divider()**：有人“typst 是不是有内置的 **html.hr** 实现来着，我现在有点找不到了”→ 答 **#divider()**。
- 131. 章节编号不能从 0 开始：有人问“每章的编号不能从 0 开始，这个如何解决啊”（未解答，可考虑手动设置/清零 **counter**）。
- 132. 章节标题各种格式 + 目录导致数字变化：<https://typst.dev/guide/FAQ/heading-various-format.html> 的方案加上目录后数字会变，0.14 和 0.15 都是这样；绕过办法是手动清零 **counter**。

4.6 页面与版式

- 133. **par** 概念容易误解——“为什么第四段没有加缩进”：关键点是 **block** 级元素会把段落切开。示例：

第一段

第二段

#block-level

第三段

第四段

#block-level

上面第一段和第二段同属一个 **par**，第三和第四段同属一个 **par**（中间被 **block** 级元素隔断）。另外 **#lorem(100)** 因为没有空行所以是 **box level** 的，需要手动加个 **parbreak** 使之成为 **par**。

- 134. 首词下沉（**drop cap**）：有人问怎么做，回复认为“像一个 **box** 或 **place** 就行的事”，找第一个词就 **split** 一下。群里未给出完整实现（待补）。
- 135. 分栏页面做跨栏图表：问“分成双列的页面怎么实现这种”（（该效果截图未包含在本地资源中））→ 答 **column balance**，但该 **PR** 还没有合并。
- 136. 页码/页眉用 **grid** 时容易被 **show** 规则污染：见第 155 条（**modern-nju-thesis** 案例）。最小复现（**#set page** 的 **header** 用 **grid**，正文里的 **set grid** 泄漏到页眉）：

```
#set page( height: 4em, width: 6em, header: grid[page #context
counter(page).display()], margin: (top: 2em),)A#{ set grid(stroke: red) [B\ C\ ]
pagebreak() [D\ E]}#pagebreak()F
```

- 137. **PDF** 元数据 **CreateDate** 不考虑本地时区？：有人提问“typst 在写入 **pdf** 元数据的 **CreateDate** 的时候，是不是没考虑本地时区”（未解答，待验证）。
- 138. 彩色书稿改黑白出版：结论是这属于 **PDF** 层面的问题而非 **Typst** 层面（“typst 层面的问题才会说 **llm** 不大可靠，**llm** 神力”），应该让出版社处理（涉及 **postscript**）。

139. 完整实验报告模板 **lib.typ**（可作为模板参考）：第 110 条的完整源码，展示了 zebraw 代码块、theorion 定理框、gentle-clues 提示框、cosmos 主题、字体回退链、页眉分割线、show "。": "."（把中文句号替换成英文句点）、主题色统一、信息表格等一整套写法。

```
#import "@preview/itemize:0.2.0" as _itemize
#import "@preview/zebraw:0.6.1": zebraw
#import "@preview/gentle-clues:1.3.1"
#import "@preview/theorion:0.6.0": *
// #import cosmos.simple: *
#import cosmos.fancy: *
// #import cosmos.rainbow: *
// #import cosmos.clouds: *
// #import "utils.typ": *

#let sans = ("New Computer Modern Sans", "Source Han Sans SC")

#let code-highlight-color = rgb("#2a61e2").transparentize(95%)
#let zcode = zebraw.with(
  background-color: luma(251),
  hanging-indent: true,
  indentation: 4,
  highlight-color: code-highlight-color,
)

#let capitalize(s) = upper(s.at(0)) + s.slice(1)

#let gc-funcs = (gentle-clues.idea, gentle-clues.info, gentle-clues.example, gentle-clues.tip)
#let (idea, info, example, tip) = gc-funcs.map(
  f => (
    body => f(
      title: text(font: sans, size: 12pt, capitalize(repr(f))),
      text(font: sans, size: 11pt, body),
    ),
  ),
)

#let (problem-counter, problem-box, problem, show-problem) = make-frame(
  "problem",
  theorion-il8n-map.at("problem"),
  counter: theorem-counter,
  render: fancy-box.with(
    get-border-color: get-quaternary-border-color,
    get-body-color: get-quaternary-body-color,
    get-symbol: get-quaternary-symbol,
    breakable: true
  ),
)

#let report(
  name: "syqwq",
  course: "IAI",
  date: datetime.today(),
```

```

tutor: "Tutor",
id: 111,
exp-name: "exp name",
grade: 2024,
body,
) = {
  set text(font: ("Libertinus Serif", "Source Han Serif SC"))

  set page(
    header: box(
      width: 100%,
      stroke: (bottom: luma(200) + .7pt),
      outset: 3pt,
      align(center, text(fill: luma(100))[报告]),
    ),
    numbering: "1/1",
  )

  set heading(numbering: "I.1.1")
  set par(justify: true)
  show "。": "."
  set enum(numbering: "(1)")
  show: _itemize.default-enum-list.with(indent: .5em)
  show figure: set block(breakable: true)
  show: zcode
  // show raw: set text(font: "Consolas Nerd Font")
  show raw: set text(font: ("FiraCode Nerd Font", "Source Han Sans SC"))
  show outline.entry: set text(fill: rgb("#21609a"))
  show link: set text(fill: rgb("#21609a"))
  show ref: set text(fill: rgb("#21609a"))
  show: show-theorion

  align(center, text(size: 17pt, weight: 600)[报告])

  let ti(a, b) = [#strong(a): #b]
  table(
    columns: (1fr,) * 3,
    stroke: luma(220) + .5pt,
    inset: 7pt,
    ti("课程名称", course), ti("年级", grade), ti("上机实践时间", date.display("[year].[month].[day]")),
    ti("指导教师", tutor), ti("姓名", name), [],
    ti("上机实践名称", exp-name), ti("学号", id), [],
  )

  line(length: 100%)

  body
  /*
  = 实验任务

```

```

= 使用环境

= 实验过程

= 总结
*/
}

/*
#table(
  columns: 2,
  align: (horizon,) * 2,
  rows: (auto, 2em),
  stroke: none,
  table.hline(y: 0, stroke: 2pt),
  table.hline(y: 1),
  table.hline(y: 7, stroke: 2pt),

  [*符号*], [*定义*],
  [$S_t$], [第 $t$ 回合的静态兵力评分],
  [$X$], [玩家投降时刻的兵力差],
  [$M_{(i\ j)}$], [兵种 $i$ 击杀兵种 $j$ 的频次],
  [$S\ e\ q$], [开局前 $T$ 步的着法序列],
  [$E_{(1)}$], [白方的 ELO 等级分],
  [$E_{(2)}$], [黑方的 ELO 等级分],
)
*/

```

140. 代码块 **figure** 自动分页会留白 → **breakable: true**: 用 figure 包 zebraw 代码块时, 如果前一页放不下, 整块代码会被推到下一页、前一页空一大片。解决: **breakable: true**, 或全局 **#show figure: set block(breakable: true)**。完整示例 (supplement: "例" + figure.caption(position: top) 把标题放上方):

```

#let code-path = "src/introduction/data_reg/data_reg.sv"
#figure(
  zebraw(
    raw(
      read(code-path),
      lang: "sv",
      align: left,
      block: true,
    ),
    line-range: (1, 24),
    highlight-lines: (
      // ..range(7),
      // (7, [bus必须声明为wire类型, 以支持多驱动信号的连接]),
    ),
  ),
  kind: raw,
  supplement: "例",
  caption: figure.caption(

```

```
position: top,
[模块data_reg] + linebreak() + code-path,
),
)<data_reg_module>
```

141. **zebraw** 的 **line-range** 不支持跳行：现在只能显示连续区间，想显示 1、4-11 行（跳过 2、3 行）做不到，除非自定义代码行显示。API 改进建议：`line-range.flatten.sort.dedup`，这样是否包含右端点就可以用 `range(1, 100)` 的参数来控制。好看的代码块包推荐见（该效果截图未包含在本地资源中）与（该效果截图未包含在本地资源中）；用 `codly` 时想去掉斑马条纹请自行看文档。
142. **typst.app** 付费与本地/在线免费的关系：不付费不影响本地或在线写 **Typst**；付费主要是扩网盘空间和一些小功能（“付费功能本身其实不太值，个人觉得适合当捐赠”）。相比 **Overleaf** 的优点是编译都在浏览器里，不会卡时间；缺点是中文字体麻烦（“中文字体太麻烦了，不如本地”、“中文字体一多超级麻烦”），建议“不要太自定义，在 **typst.app** 自带的里面选一个”。**VSCode** 本地与 **.app** 渲染对比见（该效果截图未包含在本地资源中）/（该效果截图未包含在本地资源中）。

4.7 编号与引用 / 参考文献

143. **GB/T 7714** 参考文献处理基准 **gb7714-bench**：项目 <https://github.com/YDX-2147483647/gb7714-bench>、站点 <https://gb7714-bench.netlify.app/>，“把之前参考文献处理结果展示了出来……看起来什么妖魔鬼怪都有，很多区别也不算错，可能就没必要定量打分了”（该效果截图未包含在本地资源中）。改进讨论：
 - 结果展示应做预归一化，去掉盘古之白、半角全角这些各家有自己偏好的东西；
 - **diff** 配色/方向反了：现在是“通过一堆 **diff op** 把正确的变成错的”，建议把 **diff** 的两个参数调换一下更直观；
 - 颜色语义解释：“参考 ≈ 正确 = 绿色”，且“参考 = 删除 = 删除线”，所以参考标了绿色 + 删除线（有些 **snapshot test** 工具也这样拧着标）。已记到 <https://github.com/YDX-2147483647/gb7714-bench/issues/2>；
 - 新增 `/compare/` 页面 <https://6a4a36763e9dd129725b3532--gb7714-bench.netlify.app/compare/>（该效果截图未包含在本地资源中）；
 - 归一化脚本见 `[file:result_normalize.ts]`。
144. **gb7714-bench** 趋同度评估结论（`/converge/` 页面 <https://6a4c9f90f6ad7d4e88f621f5--gb7714-bench.netlify.app/converge/>，



图 1 聊天中的效果截图

- ，PDF `[file:评估处理结果趋同程度_GB_T 7714 Benchmark.pdf]`）：
- **LaTeX** 系列最年轻的 **citeproc-lua** 反而和国标原文最接近，且是少数能从两种数据源生成完全相同结果的引擎之一（**Zotero** 自家两种 **CSL-JSON** 导出方式结果都不完全一致，反而 **citeproc-lua** 完全一致，一度以为缓存出问题）；
 - ***.json** 的效果普遍比 ***.bib** 好（**Zotero** 内部用 **CSL-JSON**）；
 - 使用 **better.bib** 的组合中，“和国标完全一致”的比例从高到低为：**citeproc-lua > biblatex-gb7714-2025 > zotero > pandoc > typst > gbt7714-bibtex-style**（“这样看 **typst** 还不算差”）；

- **citrus** 和 **gb7714-bilingual** 目前全方面垫底——因为排序主要参考“完全一致”的比例，而这俩经常其它都对、只差句点；
 - **omni-gb7714** 只支持 2015 版，**modern-nju-thesis** 能硬塞 2025 版 CSL 但 **regex** 只能匹配 2015 版，所以不好评价；
 - **bench** 不打算补 2015 语料（已有条目 id、数据源格式、处理器、样式四个变量，再加版本就太乱了）。
145. **GB/T 7714-2025** 的两处细节差异：2025 原文推荐拼音名写全拼（如 **Li Siguang**），但各家默认仍是 **Li S G**，考虑各家都没改所以 **omni** 也还没改（可用配置项改）；姓氏前缀写法变了，如 **van der** 在 2025 里推荐写在最后、写成 **v d**，而不是以前的全拼写在前面——这个好像只有 CSL 组注意到并改了。实践中各高校期刊可能因历史惯性不跟进。
146. **Typst** 侧 2015 版国标英文“等”的修法：参考 <https://typst.dev/guide/FAQ/bib-et-al-lang.html#如何修复英文参考文献中的-等> 的法一。
147. **bilingual-bibliography** 的正确用法：

```
#import "@preview/modern-nju-thesis:0.4.1": bilingual-bibliography

// 将原本的 #bibliography("refs.bib") 替换为
#bilingual-bibliography(bibliography: bibliography.with(refs))
```

完整最小复现（含页眉 grid）：

```
#import "@preview/modern-nju-thesis:0.4.1": bilingual-bibliography

#set text(font: "Noto Serif CJK SC", size: 12pt)

#set page(
  height: 10cm,
  width: 10cm,
  header: context {
    grid(
      columns: (auto, 1fr, auto),
      gutter: 1em,
      align: (left, center, right),
      [*左侧页眉*], [第 #counter(page).display() 页], [*右侧页眉*],
    )
  },
)

#let refs = bytes(
  @article{key2020,
    author = {张三 and 李四},
    title = {示例文献标题},
    journal = {某某学报},
    year = {2020},
    volume = {12},
    number = {3},
    pages = {45--50},
  }
)

#bilingual-bibliography(
  bibliography: bibliography.with(refs),
  title: [参考文献],
)
```



```
)
```

```
#pagebreak()
```

4.8 图片与绘图

148. **zap** 画电路图（**CMOS** 反相器）：完整示例（PMOS/NMOS、电源地、与非门/或非门、lbuf/lnot 等）。遗留问题：右下角 nmos 元件怎么翻转、反相器竖着放时圈里会有一条竖线。注：**zap** 已从 **GitHub** 删库，转到了 **Codeberg**。

```
#figure(
  placement: auto,
  zap.circuit({
    import zap: *
    cetz.draw.set-style(zap: (variant: "ieee"))

    node("n1", fill: false, (0, 0), label: (content: $italic("EN")'$, anchor: "west"))
    node("n2", (0, -3), label: (content: $italic("A")'$, anchor: "west"))
    lbuf("g1", (2, 0))
    lnot("g2", (2, -3))
    // lnot("g3", (3.5, 0), (3.5, -3))
    lnand("g4", (5, -0.2))
    lxnor("g5", (5, -2.8))
    lnot("g3", (3.5, 0), (rel: (-1, 0), to: "g5.in1"))
    mosfet("pmos", channel: "p", (8, 0.3), label: "PMOS")
    mosfet("nmos", channel: "n", (8, -2.3))
    vcc("power", (8, 1.5))
    ground("gnd", (8, -3.5))

    wire("n1", "g1.in1")
    wire("n2", "g2.in1")
    wire("g1.out", "g4.in1")
    wire("g2.out", "g5.in2")
    wire("g3.out", (3.5, -2.6))
    wire((3.5, -2.6), "g5.in1")
    wire("g4.out", "pmos.g")
    wire("g5.out", "nmos.g")
    wire("pmos.s", "nmos.d")
    wire("power", "pmos.d")
    wire("gnd", "nmos.s")
  }),
  caption: [
    传统测试平台结构
  ],
)<cmos_inverter1>
```

149. **blog** 场景用 **cetz** 画图：优点是内容样式统一，“不像 mermaid 可能会有跟内容样式不一致的问题”（该效果截图未包含在本地资源中）。
150. 用 **Typst** 写一整本书的实践：<https://github.com/swiftczz/roots-affixes-book>（《词根词缀的故事》）。技术路线：主体仍是 **Markdown**，用 **Python** 解析 **md** 文档再生成 **Typst** 编译；里面的 **flowchart** 渲染是自己搓的。

151. **place** 定位的 **mm** 单位与实际打印不符（虚惊一场）：**dx**: 9.64mm, **dy**: 12.76mm 实际打印出来是 13.41 / 18.95（该效果截图未包含在本地资源中）→ 最后确认是打印机问题，不是 **Typst** 的单位问题。

4.9 导出与 HTML

152. **HTML** 导出的正确心智模型：要网页就导出 **html**，“**html** 当然总是有办法正确呈现 **typst** 的原始内容”；如果不想折腾，唯一可以相信的是 **pdf** 和 **svg**。
- **html.frame** 的作用不是让你直接写 **HTML** 代码，而是把 **Typst** 里那些无法直接转换成 **HTML** 元素的内容先渲染为 **svg**，再作为 **html** 内容输出；
 - 但不一定总是灵：如果使用社区包自定义的 **block**，而它里面的东西你没有正确处理如何变成 **html frame**，那么套完 **html.frame** 可能还是导出为空；
 - 用法思路：“你觉得这个东西他就该是 **svg**，那么你就套 **html.frame**”。
153. **tinymist** 按 **target** 分别检查 / 编译 **HTML**：直接写 **html.elem** 报错时，需要切换 **tinymist** 在检查时使用的 **target**，或 **lock** 住让 **tinymist** 知道这个文档要用 **html** 编译（CLI 侧是 **typst c** 开特性支持选项）。参考实现见 <https://github.com/kodama-community/kodama/blob/main/src/include/kodama.typ#L21-L35>。背景：有人反馈“**typst** 自定义 **block** 后也能用 **markdown MPE** 插件预览，却不能用 **Tinymist Typst** 导出 **markdown**”（该效果截图未包含在本地资源中）。
154. 把 **typ** 转成 **Word**：如果只是应付报告、不需要可编辑，完全可以贴图片。

4.10 坑与 bug / 版本差异

155. **modern-nju-thesis** 的 **bib** 会污染全局 **grid**，导致页眉歪掉（该效果截图未包含在本地资源中）（该效果截图未包含在本地资源中）：根因是 **nju** 的 **show grid.cell.where(x: 1)** 命中了页面的第一个元素（不是页眉本身），所以歪了。如果这时候在正文中引用一次 **@key2020** 就可以恢复，因为第一个元素变成了那个 **ref**。相比之下 **elegant-culsc-record** 没问题，是因为 **print-bib** 在 **show** 规则之前先自己设了一个 **heading**，这个 **heading** 成为第一个子元素，把页眉挡住了。特征：只有一页也会这样（不是手动 **pagebreak** 触发），但后面的 **pagebreak** 也很重要，见 <https://github.com/typst/typst/issues/8631>。对照组复现代码：

```
#import "@preview/elegant-culsc-record:0.11.0": print-bib

#set text(font: "Noto Serif CJK SC", size: 12pt)

#set page(
  height: 10cm,
  width: 10cm,
  header: context {
    grid(
      columns: (auto, 1fr, auto),
      gutter: 1em,
      align: (left, center, right),
      [*左侧页眉*], [第 #counter(page).display() 页], [*右侧页眉*],
    )
  },
)

#let refs = bytes(
  @article{key2020,
    author = {张三 and 李四},
    title = {示例文献标题},
    journal = {某某学报},
    year = {2020},
```

```

volume = {12},
number = {3},
pages = {45--50},
}
")

#print-bib(bibliography: bibliography.with(refs))

#pagebreak()

```

156. **Typst watch / tinymist** 爆内存（500 多页练习册）：文档引用了 9 个 json（每个 500 道题），筛选逻辑叠了几层 for + if，至少 for 4500 次。讨论出的原因与对策：
 - **typst watch** 本身功能少、一般没那么吃内存，但只要有变动它就渲染一次，比如打字打一个字就自动跑渲染，渲染可以叠加导致爆掉；
 - **tinymist** 即使不 **preview** 偶尔也会自动渲染，与 **typst watch** 叠加更糟；
 - 对策：打开 **tinymist** 设置里的 **syntax only** 模式（VSCode 里开过；nvim/hx 里作者没试过也不确定能不能开）；或改用 **typst watch**；
 - 相关经验：曾有一个 **string** 太长导致 **Typst OOM** 的案例，建议优先排查有无耗内存的计算 / 超长字符串。
157. 递归枚举可变字体组合导致缓存爆炸、**swap** 到 **OOM**（白苹果）：研究可变字体时拿全量递归枚举可变字体的组合，缓存爆炸——“应该算 6191 的情况”（<https://github.com/typst/typst/issues/6191>）。（该效果截图未包含在本地资源中）
158. **harfbuzz** 对思源 TTF 65536 字形溢出导致字体失效：见第 111 条，<https://github.com/typst/typst/issues/8434>。
159. **0.14** 的 **SimSun** 方框 **bug**：见第 109 条，0.15 修复。
160. **clip** 后 **measure** 尺寸仍为未裁切值：见第 116 条，疑似版本行为变化（0.14 时正常，有猜测是 0.10 之前的行为）。
161. **Windows** 下数学 **variant style** 不可用：见第 101 条，未定位（疑似系统装了残缺字体），后来复现不了。
162. **gstd** 仓库首页中文 **README** 链接错误：<https://github.com/Myriad-Dreamin/gstd> 首页中文 **README** 的链接应该链接到 *.md，目前链接到 *.typ 了。
163. 双击 **.ttf.ttc** 后缀的疑惑：官方下载下来确实可能是 **.ttf.ttc**（清华镜像 <https://mirrors.tuna.tsinghua.edu.cn/adobe-fonts/source-han-serif/Variable/OTC/>），“应该只是打包了”；从 git 仓库的 **release** 分支里能下载到 **ttf.ttc**，镜像站镜像的也是 **release** 分支，所以同样能下到 **otf.ttc**。

4.11 工具链与生态包

164. **tinymist** 下一代 **preview**（GPU）：渲染时间控制在 <5ms，用 **canvas + webgl**，走 **vello gpu**（<https://github.com/linebender/vello>）。不会替换默认的 **preview**（还在开发状态），预计 **tinymist v0.15.4** 发布。已知短板：**vello svg** 对复杂 **clip**、**filter** 之类支持很糙（“之前 **vello svg** 是我写的，这几个部分处理的很糙”）。相关：**SVG** 的 **clip** 本来就能是任意 **path**。
165. **tinymist** 的 **SCIP** 协议适配是做什么的：不是已经适配 **LSP** 了吗？——**SCIP** 是用来覆盖 **LSP** 处理不了的 **Typst** 脚本域：例如给 **raw block** 加跳转，或者 **hover** 自己 **example raw block** 里自定义的函数、得到函数文档说明。作者目的是“把这些能力封装起来，方便其他人开脑洞开发出功能”。相关进展：生成的 **package docs** 支持了 **rich raw block**。
166. **tinymist** VSCode 插件 **typstExtraArgs** 支持 **\${workspacePath}** 的需求：想把某个 **package path** 用相对路径放到同一个 **git repo** 里管理，写死绝对路径很难受（该效果截图未包含在本地资源中）。已有 **open PR**，等待 **merge**。
167. 编辑器预览配置：

- **Zed**: 修改配置文件用浏览器预览; 服务器启动后在浏览器里手动输入 IP 地址, 或直接在 Zed 里用任务命令打开。Zed 上写 Typst 的体验还在等 GPU preview 落地 (“然后就不用依赖 vscode preview 了”); 浏览器推荐 **vieb** (比专门弄个 profile 轻量)。
 - **tinymist.scrollPreview** 命令在 **helix** 里不起作用 (提问, 未解答)。
168. 国内环境下安装 **Typst** (写进 **Dockerfile**): Typst 没法通过 **apt** 安装, cargo 编译又太慢。方案:
- **GitHub release / cargo binstall** (binstall 也是从 release 下);
 - 国内 GitHub 上不去时用 <https://mirrors.cernet.edu.cn/app/Typst>;
 - 或从 **Arch Linux** 镜像源下 typst 包 (有群友反馈拿到的是 **0.15.0**, 已经可以 **ci** 了);
 - Linux 下 **brew install typst** 尚未验证。
169. **candy: Typst 动画引擎**: <https://github.com/ViCo-Lab/candy>, 号称“可能会成为世界上第一个工业级 Typst 动画引擎”。项目在极早期, 需要折腾, 可从 **CI** 拉取预构建二进制和 **Typst** 包, **0.1.0** 还没发布。
170. 私有包仓库不可行: **import** 除本地文件、**@preview**、**@local** 之外没有 **http** 地址等方式 (“remote 好像不行, 得魔改一下 typst”), 公司内网部署私有包仓库只能让同事自己安装到 **@local**。
171. **typst-doc-cn/guide** 的 **text-shadow FAQ**: PR <https://github.com/typst-doc-cn/guide/pull/170>, 预览 <https://deploy-preview-170--luxury-mochi-9269a9.netlify.app/FAQ/text-shadow.html>。计划把 SVG 方法放到“简单方法” (多层 **place** 法) 下面。
172. **AI 写 Typst 的实践与 skill**: 现在大规模报告都让 agent 写成 Typst 直接看 PDF——“一种介于 md 和 html 之间的优质报告格式”; 让 agent 写时要 **@local import** 一下、**show** 一下, **style** 不需要它调。但“让现在的 AI 直接写还是会出错, 得跟他说让他加载一下相关的知识才行”; 若它不主动读 **skill**, 需要显式引导。已有 **skill**: <https://github.com/Myriad-Dreamin/tinymist/blob/main/.codex/skills/typst-writing-document/SKILL.md> (纸夜写的)。也有实践反馈: 把公司后端文档导出服务换成 Typst, “我就跟后端说了注意字体, 然后扔给他一个产品经理用 word 写的示例文档, 一天时间模块就写好了”——Typst 本身有设计成适合生成模板文档的。
173. **Typst 0.15 大版本发布**: 知乎文章《Typst 0.15 大版本发布, 应该能替代 LaTeX 的新时代排版系统》<https://zhuanlan.zhihu.com/p/2054614977374376303>。
174. 其它零散问答:
- LaTeX 的 **fancytooltips** 包是干啥的, Typst 有类似的存在吗? (未解答)
 - **itemize** 列表数字后标点不挤压的问题修了没? (提问, 未解答)
 - 编号对齐“上面的正确、下面的出错” ((该效果截图未包含在本地资源中)) ——开了 **list** 编号居中, 最初以为行为不一致, 后来发现自己看错了, “上面那个确实也居中了”, 是统一的居中行为。
 - 想把 **typ** 变成 **Word**: 不需要可编辑完全可以贴图片。

5 第四部分 群聊精选 (2026-06-27 07-01)

5.1 绘图 / 渐变 / 曲线

200. **cetz** 径向渐变水滴效果 (代码—效果): 用 `gradient.radial` + 参数曲线 `line(...)` 画水滴, `rotate(-90deg, reflow: true)` 旋转。是“颜色渲染做出类似水滴”的实现范例。((该效果截图未包含在本地资源中))

```
#let tear-drop = canvas(length: 10em, {
  import draw: *
  let water-gradient = gradient.radial(
    (rgb("fafafa00"), 0%), (rgb("f3f9fc77"), 15%),
    (rgb("92bddaff"), 60%), (rgb("2870aebb"), 100%),
    center: (50%, 50%), focal-center: (25%, 30%), focal-radius: 2.5%)
  let f(t) = { /* 6cos2t-3cos4t 等参数曲线 */ (x, y) }
  let points = for i in range(401) { ... }
  line(..points, close: true, fill: water-gradient, stroke: none)
})
```

201. **genealotree** 遗传系谱图包: <https://typst.app/universe/package/genealotree>, 可加边更好读。((该效果截图未包含在本地资源中))
202. **label** → **href** 转换技巧: 发现可把 `label` 变成 `href`, 用来生成 `rss feed` 或抽 `str` 塞进 `html.img/html.link` (“可能会稍微有点慢”)。((该效果截图未包含在本地资源中)) 同一处)

5.2 数学排版

203. **plus.minus** 符号: `/typ $plus.minus$` 打出“上面+下面-”。((该效果截图未包含在本地资源中))
204. **math.cancel** 取消线: `#set math.cancel(stroke: teal) $ cancel(x) $`, `#set math.cancel(background: true)` 加底色。0.15 已有。
205. 艺术大括号 / **overbrace** 在表格里: `$overbrace("foo" "bar")$` 即可; 放进表头合并单元格用 `place`, 列内用 `h(1fr)` 对齐。若想要原书“两头窄中间宽”的艺术括号, 得 `rotate+scale` 或自绘字形/curve/svg。
206. **metadata()** 是 **content** 类型: `#{1 + metadata()}` 报错 `cannot add integer and content`。 `metadata` 包裹的是 `content`, 不能直接和整数相加——这是类型陷阱。

5.3 HTML 导出 / 自定义属性

207. 给 **HTML** 导出元素加 **ARIA** / **role**: `show html.elem.where(tag: "a"): set html.elem(attrs: (aria-label: "Back to content", role: "doc-backlink"))`。坑: 不能写 `show html.a: set html.a(aria-label: ..., role: ...)`——那样 `label` 会坏掉; `aria/role/dataset`- 这类属性只能经 `show rule` 写到 `typed html` 上, `std.link` 也不能直接写 `class`。
208. **html.span** 写 **style** 锁死 **backend**: `html.span(..., style: ...)` 可行但把导出 `backend` 锁死 **HTML**; 可 `if "html" in dictionary(std) { html.span(...) }` 做条件兼容。

5.4 show 规则 / context

209. **show** 闭包自带 **context** (关键): 嵌套样式想继承外层文字颜色, 直觉要写 `context`, 但其实 `show rule` 的闭包已处于 `context` 中, `context` 可删:

```
#show strong: it => underline(text(fill: text.fill, it)) // text.fill 自动取外层颜色
```

原写法 `#show strong: it => context { underline(text(fill: text.fill, it)) }` 的 `context` 是多余的。这是让 `#underline[ABC#strong[DEF]]` 里 `strong` 保持红色 (与 `underline` 同色) 的要点。((该效果截图未包含在本地资源中))

5.5 引用 / 交叉引用 / 大纲

210. 按 **label** 前缀给 **ref** 上色 (GitHub 风):

```
#show ref: it => {
  if str(it.target).starts-with("thm:") { highlight(fill: green, it) }
  else if str(it.target).starts-with("note:") { highlight(fill: yellow, it) }
  else { it }
}
```

211. 自定义 **outline.entry** + 跳转: `#show outline.entry: it => link(it.element.location(), it.indented(it.prefix(), it.inner()))`。it.prefix 即为 numbering 文本。((该效果截图未包含在本地资源中))

212. **numbly** 做 “Chapter/Section” 前缀编号 (极好用):

```
#import "@preview/numbly:0.1.0": numbly
#set heading(numbering: numbly("Part {1:1}.", "Chapter {2:1}.", "Section {3:1}."))
#set outline(indent: n => calc.max(0, n - 1) * 2em)
#outline()
```

除多跑几次 numbering 外十分优雅。((该效果截图未包含在本地资源中))

Contents

Part 1. aaa	1
Chapter 1. bbb	1
Section 1. ccc	1
1.1.1.1 ddd	1

图 2 聊天中的效果截图

)

5.6 标题编号 / counter

213. 获取当前标题编号: `context counter(heading).get()` 返回各级编号的 array, 本身没问题; 萌新报错的真正原因在别处 (不是 counter 调用错)。需与 `set heading(numbering: "1.")` 搭配。

5.7 变长参数

214. 变长参数取长度: `numbering: (..xs) => ...` 可用 `xs.pos() + len(xs.pos())` 拿到参数列表长度与各级编号 (萌新卡点, 已解决)。

5.8 表格

215. 表头背景色两法: `show table.cell.where(y: 0): set table.cell(fill: teal)` (table guide 官方), 或 `#table(columns: 2, fill: (x, y) => if y == 0 { teal }, [A], [B], [C])`。无 header 时无需设置。

216. 表格奇偶/粗细边框: `stroke: (x, _) => (left: if x > 0 { a } else { b }, rest: b)` 实现“左列粗、其余细”。([img:待查])

217. 表头合并单元格 + 艺术括号: 用 `place` 进合并单元格, 列内 `h(1fr)` 对齐。

5.9 图片导入

218. 导入 **BMP**: `typst` 仅支持 `rgb8`, 而 **BMP** 是 **BGR**, 需 `read("x.bmp", encoding: none)` 取 bytes 砍头后用 `image.decode` (第一个 **format** 参数必须手动指定), 再手动转 **BGR**→**RGB**; 最简单还是转 **PNG** (`imagemagick/nix shell nixpkgs#imagemagick --command magick input.bmp output.png`)。写插件也行但没必要。

5.10 列 / 页面

219. 列平衡 (**dev/0.16**): `#set page(width: 300pt, height: auto) #rect(inset: 0pt, stroke: yellow, columns(3, balanced: true, lorem(40)))` 已在 main 分支支持, 0.16 才稳定。([img:322] 展示 dev 版 columns 平衡)
220. 自适应紧凑页面: 探针常用 `#set page(width: auto, height: auto, margin: 1em)` 让渲染结果刚好包住内容。

5.11 typst eval / 作用域

221. **typst eval** 作用域: 第三方包函数不能凭空注入, 必须 `import` 或传 `--scope`; 否则报 `scope` 错误。 `typst eval` 失败退出码 1 (0.15 pr8623)。

5.12 工具链 / 生态包

222. 海獭 **haita 0.3.0-rc1**: 纯 Typst HTML 文档工具, 更新样式/字体/代码块显示, 提供 Pagefind 支持, <https://github.com/wensimehrp/haita>。
223. **0.15 changelog** 要点: NCM 升 8.1.1 修复 calligraphic letterforms (pr8552, `@text.stylistic-set[stylistic set 6]`); `math.lr` 内对齐点回归修复 (pr8566); `math.op` 垂直错位修复 (pr8546); 多页 `list-number-align/marker-align` 空隙 bug (pr8649); PNG/SVG 多页错误仅最后一帧报 (pr8618); SVG inline pretty print (pr8535); watch server Content-Type 扩展 json 等 (pr8650)。
224. **tinymist**: 有 profiling 功能抓性能热点 ((该效果截图未包含在本地资源中)), Apache-2.0 协议; F1 “typst pin the main file to the currently open document” 固定入口避免 `template` 入口不触发报错; `tinymist.typstExtraArgs` 不被 profiling 支持。
225. **typst.ts** 升 **0.15**: 正式 release v0.7.0 到 0.14.2, v0.8.0-rc3 才有 0.15-rc1; 可 patch。
226. 绘图/公式包动态: `cetz 0.5.2` (canvas/draw)、`lilaq` (公式绘图)、`zebraw 0.6.3` (代码块, 长行折行待改进)、`molchemist` (分子结构式, 支持 SMILES, 但遮盖关系不行, 想接入 `typsium`)、`touying` (slides, 压测会不编译)。
227. **typst** → 视频: `tanim / Candy` (静态链接 `typst+ffmpeg`, x264 编码, 帧间缓存+多线程, 3.2K 60FPS 两分半跑通内存 6G, 吊打 `Manim`) / `TOSS` (pku-typst, 纯前端) / `Janim` (typst 结合)。

5.13 中文字体 / 中文社区

228. 中文社区 **FAQ** 字体设置页 (`typst-doc-cn/guide PR180`): 推荐思源宋体, 链接 MirrorZ/CERNET 镜像; `typst fonts --variants` 会输出字体文件路径; 删除“不支持可变字体”过时内容。本地用思源而非其它字体, 因其它字体遇 `cuti` (粗体缺字重)、`latin-in-cjk` 问题。Noto CJK 在 `typst.app` 几乎唯一能直接用, 其中宋体更常见。

5.14 文献著录 (GB/T 7714) 深度专题

229. 多文献脚注拆分: GB/T 7714 note 样式同一处引多篇应有多个脚注。omni 默认支持; 依据 hayagriva500。omni footnote-ibid 默认 true, 设 `footnote-ibid: false` 复刻 2025 CSL; 准备改成 auto 按 15/25 版本感知。
230. 多文献库包: 原生路由写法之外, 社区有 alexandria、pergamon 两个多文献库包; omni-gb7714 已支持原生路由 (0.14 仍可用)。
231. “.] 是单个 token” 导致 LLM 写 **gb7714** 实现差句点: `gb7714-bilingual / citrus / citum` 近年新创且大量用 AI, 均出现“差个句点”的一致问题; 怀疑 LLM tokenizer 把] . 与] 认同 (类似普通人看 - 与 -)。链接 tokenizer playground。
232. omni 自定义 **driver** 机制 (YDX 详解, 极有信息量): 处理流程 `_tokenize` → `_parse` → `_resolve` → `_smart-join`。
- `{ }` verbatim 原样输出, 不做标点校正; 条件组 `?<..>(任一非空) / &<..>(全非空) / <..>(普通)`; 别名链 `A|B`; 软空格按排版规则决定显隐 (贴右标点/全角前后不显)。

- **smart-join** 用 **buffer**: 规则 1 空 **token** 两侧元素间符留、元素内绑定的符删（按国标标点层级：句点是元素间连接符，: // （）在元素内）；规则 2 元素前导符是词法概念不滞留到渲染期（**seen-data** 由首个数据 **token** 触发，空也算）；规则 3 首个内容前各元素按各自规则。
- **_resolve-separator**: a. 句点去重按左末字符（**et al. West[M]** 不重复句点）；b. 软空格三类 **hug** 判定（左贴右标点或右贴左标点则不加空格，全角不额外空格）。
- 与 **biblatex** 的 **\setunit** 缓冲机制（标点存 **\blx@unitpunct**，下个 **\printfield** 输出时才 **flush**，空字段则被覆盖）几乎同构。

5.15 WYSIWYG 编辑器 / 笔记软件

233. **tylina / tyx**: **svg-based WYSIWYG** 编辑器讨论。**content** 节点可编辑、**text span** 选中、上下箭头在可编辑 **content** 间切换；问题多（退格报错、白屏、跨文件跳转不行、**backspace** 误选全文、公式编辑框高度小）。仅工具介绍。
234. **InkyCap**（**Codeberg** 开源笔记软件，**typst preview**，未做 **LSP** 集成）；**obsidian + tinymist** 也是笔记方案。

5.16 待补 / 未解

235. 汉字头顶注拼音：群里未给出方案（“**typst** 咋注音来着”/**taipst/**），待后续补。**fontfeatures** 或 **ruby** 类包可能相关。
236. **Source Han Sans HC** 疑似炸（**issue 8434** 再现，与 **c00111** 同因）：**ttc 65536** 字形 **harfbuzz** 溢出 → 字体读取失败，换 **OTF/OTC**。

6 第五部分 群聊精选（2026-07-01 前后）

6.1 字体与上下标渲染（Inter super/sub 大案）

300. **Inter** 字体 `#super[[123]]` 渲染异常的起因与初步排查。现象：设置 **Inter** 字体后，纯数字上标 `#super[[123]]` 渲染奇怪（上标变小/错位），而 **LaTeX** 能正确渲染；且“只在纯数字情况下”异常。初步怀疑与字体自带的上标度量（`sups`）有关，参考 <https://gap.zhtyp.art/#mix-super-metric>。临时绕过：把 `baseline`、`size` 定死，不用字体提供的数据：

```
#set super(  
  baseline: -0.4em,  
  size: 0.6em,  
  typographic: false,  
)
```

讨论演进：先怀疑华文宋体 **STSong**（其元数据声明上下标尺寸应为普通字的 1/10，见 <https://gap.zhtyp.art/#stsong-super>）→ 测试后发现“不是基线的锅”，最小修复方案是 `typographic: false`。

301. **sups** 字形回退策略猜想：字体可能只为纯数字提供了上标专用 `sups` 字形，不提供中括号的，于是 `123` 调用 `sups` 字形，`[123]` 调普通字形。进一步的怪现象：`[English 123]` 正常上标而 `[123 English]` 不正常；`[1]` 和 `[ 1]`（数字前加空格）也不一样。

302. 版本回归定位：该问题 **v0.13.1** 正常，**v0.14.0** 开始出错——**0.14** 当时修了另一个上标 `[1]` 飞起的问题（见 <https://gap.zhtyp.art/#cite-number-flying>），可能修坏了。三个版本对比结论：**0.14** 渲染前括号失误而后括号正确；**0.15** 保证了前后括号一致（一起错）；**0.13** 疑似完全正确，但肉眼看是靠忽略字体自带 `sups` 字形实现的。另发现“在结尾有数字且长度过长的情况下尾括号会出错”。

303. 最小复现代码（**/typ** 发到群里渲染）：

```
// 刚装了个Inter（可变版本）  
#set text(font: "Inter")  
- A#super[[1]]  
- A#super[1\]]  
- A#super[ 1\]]  
- A#super[\[1]  
- A#super[\[1 ]
```

- A^[1]
- A^{1]}
- A^{1]}
- A^[1]
- A^[1]

▲ 代码实测（本机 **typst** 渲染）

304. 用 **OpenType features** 直接测 `subs` 字形：

```
#text(font: "Inter", features: ("subs",), "[ ]aA{ }[ ] [ ] [ ] [ ]")
```

[]aA{ } [] [] [] []

▲ 代码实测（本机 **typst** 渲染）

```
#text(font: "Inter", features: ("subs",), "[ ]aA{ }[ ] [ ] [ ] [ ]")
```

[]aA{ } [] [] [] []

▲ 代码实测（本机 **typst** 渲染）

结论：无论上标还是下标，都出现 `sub[ [ ]`、`sub[ 1 ]`、`sub[ ] ]` $\neq$ `sub[ [1] ]` 的情况；圆括号与中括号表现一致地出错。

305. **harfbuzz** 对照实验：怀疑还与科学下标 `sinf` 特性有关。用 `harfbuzz 14.2.1` (<https://github.com/harfbuzz/harfbuzz/releases/tag/14.2.1>) 的 `hb-view` 对 `InterVariable.ttf` 分别以标准字形、`--features subs`、`--features sinf` 渲染 `Aa1[]{}()` 对照。OpenType 特性列表参考 https://en.wikipedia.org/wiki/List_of_typographic_features。后经确认与 `sinf` 关系不大。

306. 可变字体 vs 静态字体变量：Discord 上有人指出 `typst.app v0.15` 的 `Inter` 用的是可变版本，`v0.14` 及之前是静态版本。本地用 `Inter 4.1` 静态版本重测，行为仍有诡异之处。

307. 中途岔出的语法知识点——**markup** 模式下分号消失：`A#sub[\[]#sub[1]#sub[\]]`；`A#super[...]` 里中间的分号没渲染出来。解释：分号用于终结前面的代码模式；前面刚好不是 `content` 时会吞掉。可以用 `;;` 或 `\;`。

```
/typ A; A\;
```

```
/typ A; A;
```

▲ 代码实测（本机 `typst` 渲染）

正常写文本时分号不会消失。

308. 真相：**calt** (**contextual alternates**) 连字替换是罪魁祸首。关键实验：

```
#set text(font: "Inter")
#super("[1e]")#super("[e1]")
#set text(features: (calt: 0))
#super("[1e]")#super("[e1]")
```

```
[1e][e1] [1e][e1]
```

▲ 代码实测（本机 `typst` 渲染）

关闭 `calt` 后 `[1e]` 与 `[e1]` 表现一致、恢复正常。`calt` 默认启用，有的字体用它把 “ $\Rightarrow$ ” 替换成 “ $\Rightarrow$ ”；`Inter` 的 `calt` 设置（上标环境下的替换）有问题。文档：<https://typst.app/docs/reference/text/text#parameters-features>

309. 最终定论与 **issue** 归档：空格的异常是空格没有 `sup`s 码位导致回退；其余异常是 `calt` 连字替换的问题。对照验证：`Palatino Linotype`（提供圆括号上标）、`Libertinus Serif`、`STIX Two Text`、`Noto Sans/Serif` 的圆括号都工作正常；中括号 `sup`s 支持几乎只有 `Inter` 一家。提交的 `issue`：<https://github.com/typst/typst/issues/8690>（后自行关闭）。另评估过是否上报 `Inter` 仓库（<https://github.com/rsms/inter/>，上次提交 2024 年 11 月，短期内解决希望不大）。验证用的完整对照代码：

```
#set page(height: auto, width: auto, margin: 1em)
#set text(5em, font: "Inter")
A[
  #super("[1]")#super("[1]")
  #[
    #set text(features: (calt: 0))
    A[
      #super("[1]")#super("[1]")
    ]
  ]
#set text(red.transparentize(50%))
A[
  #super("[1]")#super("[1]")
  #context v(-measure[A].height, weak: true)
  #set text(features: (calt: 0), green.transparentize(50%))
  A[
    #super("[1]")#super("[1]")
  ]
]
```

(效果: , 红色/绿色半透明叠加对比 `calt` 开关差异)

310. 得意黑不是斜体: 把汉字写在气球上拉斜只是“伪斜体”(shear)。oblique 与 italic 的区别: 艺术字体可以设计成斜的, 但正经排版不该把正体字体直接 shear 成斜。得意黑的字框仍然正立, 真正的斜体是把字形机械旋转约 10° ——得意黑是一种具有“倾斜视觉效果”的正体字体。

311. 竖排蒙古文: 群内提到 Typst 做竖排的困难, 参考 <https://devinz.org/mongolian-vs-chinese.html>; 相关字体作者: 吉日本图、にしがず (日本公司做蒙古文字体的缘由)。

6.2 数学排版

312. **math** 模式冒号间距像二元运算符的解法: colon 和 : 在 math 里的间距都像“比例”运算符, 想要文本冒号行为, 用 `math.class("punctuation", ...)` 重设类别:

```
$ A colon B wide wide A : B \ A#sym.colon B wide wide A ":" B $
```

$A : B$ $A : B$
 $A : B$ $A : B$

▲ 代码实测 (本机 typst 渲染)

```
$ f class("punctuation", :) A -> B $
```

$f: A \rightarrow B$

▲ 代码实测 (本机 typst 渲染)

最终推荐写法 (可复用、还可 `#h(-1pt)` 微调):

```
#let colon = math.class("punctuation", ":")  
$A colon B wide wide A : B \  
A ":" B $
```

$A : B$ $A : B$
 $A : B$

▲ 代码实测 (本机 typst 渲染)

313. **cases** 里分式太小: 不建议全局改, 在需要的地方加 `display` (行内分式变小是同样机制)。官方 FAQ: <https://typst.dev/guide/FAQ/dcases.html#> 如何让-cases-里面的分数-公式显示成-display-形式

6.3 文本、断行与两端对齐

314. 行末破折号断行修复 (**PR** 接力): 经过中国人 Ri-Nai 和俄国人 sicikh 的接力 PR, 行末破折号能正常用了: <https://github.com/typst/typst/pull/7376> 与 <https://github.com/typst/typst/pull/8131>。测试代码 (稳定版尚未包含修复, 需 /typdev 开发版):

```
#set par(justify: true)  
#block(width: 9em, stroke: (right: green)) [  
  娜拉走后怎样? —别人可是也发表过意见的。一个英国人曾作一篇戏剧.....  
]
```

娜拉走后怎样? —
别人可是也发表过意
见的。一个英国人曾
作一篇戏剧.....

▲ 代码实测 (本机 typst 渲染)

(效果: 稳定版; 开发版 typst version: main.2026-08-04.540db57)

6.4 页面与版式

315. A3 试卷双栏中间竖线：不用 `stroke`，放页面背景上：

```
#set page(  
  ..  
  columns: 2,  
  background: place(  
    center + horizon,  
    rect(width: .., height: ..),  
  ),  
)
```

316. 像素字体时刻表作品：在线项目 <https://typst.app/project/rHQxrdgQ3RL3a5dkm9cHQ8>。技巧：字体是 6x11 像素字体，字号设成 16mm 恰好 $1\text{px} = 1\text{mm}$ ；LED 圆点不用字体而用 `curve` 画出来盖在上面，以便控制每个圈的大小和形状。（群文件 [file:test-timetable-3.pdf]）

317. **radius: 50%** 渲染异常（未解决 **bug** 现象）：同一文档前 2 页正常、后 2 页出现图片未裁剪问题，**radius: 50%** 的结果不是圆形，“到了上面又变成圆的了”。（效果：（该效果截图未包含在本地资源中）（该效果截图未包含在本地资源中））疑似与增量编译/布局缓存相关的渲染不一致，群内未查明根因。

6.5 语言特性（**state** / **context** / 函数）

318. 往 **state** 里存函数必须套一层：新手问题——`state.update(red-answer)` 后 `state.get()` 拿到的不是原函数。根因（官方文档 <https://typst.app/docs/reference/introspection/state/#definitions-update>）：`update` 收到非函数值时直接设为该值；收到函数时，该函数会被当作“更新函数”，接收 `state` 的旧值并返回新值。所以想把函数本身存进 `state`，必须再包一层传给 `update`。原始复现代码：

```
#let Set_Func_AnswerWrap(func) = {  
  state("s_AnswerWrap").update(func)  
}  
#let red-answer(x) = text.with(fill: red, size: 0.93em, tracking: 1pt, font: "LXGW WenKai")  
#let red-answer2 = text.with(fill: red, size: 0.93em, tracking: 1pt, font: "LXGW WenKai")  
#{  
  Set_Func_AnswerWrap(red-answer) // 将函数存入状态  
  context {  
    let x = state("s_AnswerWrap").get()  
    if (x == red-answer) { [yes] } else { [不一致] }  
    x()[红色]  
  }  
}
```

不一致红色

▲ 代码实测（本机 typst 渲染）

另一个困惑点：`red-answer2`（`text.with(...)` 的结果）不能放在 `#set` 后面——只有 **element function** 可以放 `set` 的后面。

319. **context** 嵌套时 `x.get()` 返回值语义演示：

```
// #set page(paper: "a3")  
#let txt1 = state("color", red)  
#let txt2 = state("color", blue)  
#let query(x) = { x.get() }
```

```
#context {
  let dest = query(txt1)
  context { assert.eq(dest == txt1.get(), true) }
}
#let query(x) = context { x.get() }
#context {
  let dest = query(txt1)
  context { assert.eq(dest == txt1.get(), false) }
}
#context {
  set text(fill: txt1.get())
  [ 这是红色的 ]
  set text(fill: txt2.get())
  [ 这是蓝色的 ]
}
```

(效果:，“正常的黑色”说明直接 set fill 用 state.get() 在非 deferred context 下拿到的是 state 对象本身而非值)

6.6 字典、参数与模板设计

320. **stroke** 字典回退默认值的坑: 想实现“每个键: 传入有则用传入的, 否则回退到自定义 base-stroke, 再否则 Typst 默认”。问题: **stroke: black+1pt** 这种写法会自动调用 **stroke** 构造函数, 导致 **thickness/paint** 之外的键全是 Typst 默认值, 无法区分 **.cap == auto** 是调用方显式传的字典键还是构造默认。结论: Typst 自带 **element** 函数不存在此问题, 因为 **auto** 有 **inherit** (样式链继承) 效果; **cetz** 的 **draw** 模块由 **set-style** 函数处理 **inherit**; 最终把 **cetz styles** 里的内部函数 **_stroke-to-dict** 包一层 (复制粘贴出来) 解决。
321. 模板参数设计观点 (有明确论点): 模板开发者应尽量不列一堆参数让最终作者传 (尤其 Typst 本来就有的参数) —— 编辑器里这些参数的文档、补全都很差, 一个模板一套命名规则, 且模板作者没精力测试所有参数组合。理想情况是模板只用 **set** 和 **show set**, 最终作者在 **#show: template** 下面自己 **set/show** 定制。现实妥协: 模板工程里甩一个全局字典配置, 改参数直接改字典。注: **#show/#set** 只能作用于内置元素, 开发者自定义函数做不到。

6.7 列表与编号

322. + 列表中间插 **#grid** 导致 **enumerate** 编号重置 (显示 1, 1):

```
+ 第一项
#grid(
  columns: (1fr, 1fr),
  [左],
  [右],
)
+ 第二项
```

- | | |
|--------|---|
| 1. 第一项 | |
| 左 | 右 |
| 2. 第二项 | |

▲ 代码实测 (本机 typst 渲染)

解法: 把 **grid** 缩进到列表项内部即可 (不缩进会被解析为独立内容打断列表)。

323. **enum** 多级悬挂缩进问题 (未完全解决): 想让各级 **enum** 都有常规段落对齐,

```
#show enum: it => {
  set enum(indent: 2em)
  set par(hanging-indent: -3em)
  it
}
```

只对第一级生效，二、三级仍有缩进。群内提示“是不是需要 level 1 / 保留 level 1”（即用 `enum.where(level: 1)` 类选择器限定），未见完整验证。

324. **heading** 编号用全角括号：

```
#set page(width: 120pt, margin: 10pt)
#set heading(numbering: " [1] ")
= 犹可说也不可说也
```

6.8 参考文献与 GB/T 7714

325. **omni-gb7714 v0.0.730** 的国标对齐细节：GB/T 7714 给的版次示例为“ed”（不带缩写点），omni 对齐了国标原文；句点有去重逻辑（英文里句子以 etc.、a.m. 结尾时句号与缩写点合并）。为不完整文献加判定：omni 中一条文献按 block 渲染（block = 若干 field 组成的区块，如期刊文献的“年期卷页”；连续出版物的起止区间用 serial-block 调用），若 `block ≤ 1` 则不显示末尾句点——对真实语料无负面影响，还能对上国标里大量不完整语料。基准测试看板：<https://6a6b6c80d511a21bba639ec4--gb7714-bench.netlify.app>、<https://6a6d65675920045f5bd0a0cf--gb7714-bench.netlify.app/diagram.pdf>（效果：（该效果截图未包含在本地资源中）

[gb7714.zhtyp.art](https://github.com/omni-gb7714)

各种处理结果与国标原文一致的条目数量

（仅计完全一致的；比较时忽略大小写和标点符号编码方式差异）

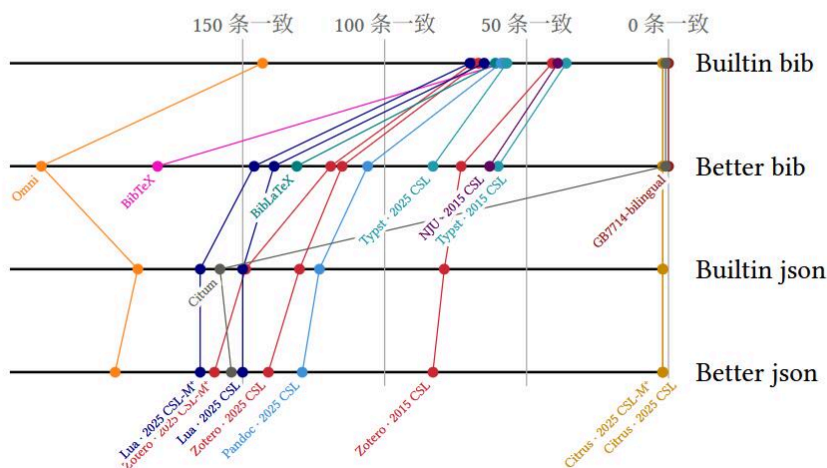


图3 聊天中的效果截图

）。omni 加了 JSON 支持后，better json 表现超过 lua 2025 csl-m（效果：（该效果截图未包含在本地资源中））。上游语料纠错渠道：<https://www.github.com/zotero-chinese/styles/discussions/693>

326. **GB/T 7714—2025** 换版与勘误：中国标网电子版换新版本，diff 用 <https://soft.rubypdf.com/software/diffpdf> 制作（[file:diff.pdf]、[file:GB_T 7714—2025《信息与文献 参考文献著录规则》-2026 年夏勘误.pdf]）。吐槽点：勘误中“糸”的旧字形竟然是贴的图片（方正书版做不到？）（效果：（该效果截图未包含在本地资源中））；弯引号改成直引号“反而更错了”（效果：

ealing nature's synthetic potential t
 } Chemical Biology, 2013, 8 (3):

图4 聊天中的效果截图

); 一字线原本符合国标标点用法 (效果:

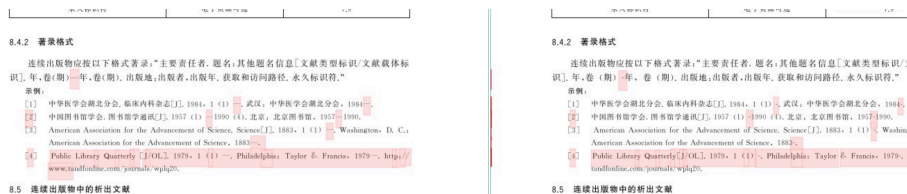


图5 聊天中的效果截图

)。2025 版把 Revised 仍缩写为 Rev. 但未词不加缩写点, 疑似“连续多个缩写词最末一词不加缩写点”(效果: (该效果截图未包含在本地资源中) 2025 / (该效果截图未包含在本地资源中) 2015 / (该效果截图未包含在本地资源中) 2005)。

6.9 工具链与生态包

327. **tinymist** 导出 **bug (win10)**: 路径下存在与 .typ 文件同名 (不含后缀) 的目录时, 导出失败并报错。(效果: (该效果截图未包含在本地资源中), 目录结构 (该效果截图未包含在本地资源中))
328. **tinymist** 预览与编辑功能特性: SVG 含 foreignObject 时 Typst 导出有问题, 但 tinymist 预览正常——因为 tinymist 相当于在浏览器里打开 SVG (甚至能播放 gif 动画), 说明 tinymist 对渲染有不少私改。seasides (touying skill + 编辑器, <https://github.com/touying-typ/seasides>) 支持 markdown 粘贴为 typst (表格、代码转换成功); ChatGPT 网页输出可直接粘贴含公式; deepseek/豆包网页版公式识别不出——因为 deepseek 的 katex 没加 copy-tex 扩展, 可装浏览器插件直接复制 raw tex source: <https://github.com/kokic/copy-tex-extension>; kimi 输出不带 LaTeX 源码则没办法。
329. 绘图包动态: gribouille (<https://github.com/mcanouil/gribouille>, ggplot2 风格) 更新 0.6; lilaq (matplotlib 风格) 也是 0.6。群内画风评价: lilaq 像 matplotlib, gribouille 像 ggplot2。
330. **typed-scores** 乐谱包: <https://typst.app/universe/package/typed-scores> ——把音符符号字体转成了 SVG, 内部有一个“恐怖的巨大 lib.rs”, 绝大部分代码是手搓 parser。
331. **touying** 相关坑: tblock、pause 是 touying (theme.stargazer) 的功能; tblock 的框始终显示、与 pause 疑似冲突——“这个容易导致 layout 五次不收敛然后显示结果异常”(Typst layout 迭代上限为 5 次, 不收敛会输出异常结果)。顺序编码制参考文献加动画的问题: 简单的 frozen-counters 对 #bibliography() 无效, 见 <https://github.com/touying-typ/touying/issues/415>。
332. 模板与框架发布: Haita 0.4.0-rc1 纯 Typst 文档框架 (<https://github.com/wensimehrp/haita>, 更新日志 <https://wensimehrp.github.io/haita/changelog.html>, 新增 Favicon、数学公式可复制——做法是先写 \$ 再改成三反引号围栏保留 LSP 补全); 北大学位论文模板 <https://github.com/chuxinyuan/pku-thesis-pass> (模板评审经验: 别人贡献的官方 logo 被问授权许可, 只能改成灰色占位框 + 参数允许用户自定义导入)。
333. 让 .typ 伪装成 .tex 用 latexmk 编译: 在 main.tex 旁放 main.typ, 运行 latexmk 即从 main.typ 输出 main.pdf:

```
$ cat latexmkrc
# 参考 https://mirror.ctan.org/support/latexmk/example_rcfiles/tex4ht-latexmkrc
$dvi_mode = 1;
$pdf_mode = 0; # 不让 lualatex 输出 PDF, 并允许调用 typst
```

```
# 为保持目录、交叉链接, 选择让 typst 直接输出 PDF, 而不是 LaTeX pdfpages \includepdf
$latex = 'lualatex --output-format=dvi --shell-escape';

$ cat main.tex
\documentclass{article}
\usepackage{shellesc}
\begin{document}
\ShellEscape{typst compile main.typ main.pdf}
If you see this line in the PDF, you should recompile with latexmk.
Refer to README.txt for details.
% That line also prevents _No pages of output_, so latexmk will treat the
compilation as successful.
\end{document}
```

依赖安装: `tlmgr install scheme-minimal collection-latex luatex latexmk + curl -fsSL https://install.typst.community/install.sh | sh -s 0.15.1`。

334. **LLM 写 Typst 的资源与技巧**: 提示词/skill 资源列表 <https://ydx-2147483647.github.io/best-of-typst/#docs> (含专门用 typst 训练过的 LLM); tinymist 开发者 Myriad-Dreamin 的 skill: <https://github.com/Myriad-Dreamin/tinymist/blob/main/.codex/skills/typst-writing-document/SKILL.md>。自制公式 skill 思路: 把官方文档的符号表 + math DSL 语法拆出来喂给 LLM 炼成 skill。给 Agent 测试题 (introspection + layout 综合题): 编写高度为 auto 的文档, 多个 title 各介绍一个拉普拉斯变换推导, 开头用 introspection 列出标题及其与页顶距离 (厘米)——deepseek flash 0731 首次编译成功用了第九次 (效果: (该效果截图未包含在本地资源中)) (该效果截图未包含在本地资源中))。deepseek 写质数表格会用 `n % k == 0` 而非 `calc.mod`, 报错后会尝试修正 (效果: (该效果截图未包含在本地资源中)) (该效果截图未包含在本地资源中))。反方观点: 元宝等模型经常把 markdown 表格语法当 typst 语法 (效果: (该效果截图未包含在本地资源中))，“LLM 写 Typst 正确率高于 LaTeX”的说法存疑; 折中方案: 让 LLM 输出 markdown 再 Pandoc 转换, 或先喂 100 个 doc.typ 再写。
335. **tinymist wysiwyg / Nodes mode** 设计讨论: 渲染结果上直接编辑 (类似公式编辑器的就地编辑)、侧边栏右下角节点模式、breadcrumb 显示“当前在哪个函数里” (应对 `#[xxx #[yyy #[zzz]]]`) 深层嵌套, 可在编辑区顶部/光标旁显示 (效果: (该效果截图未包含在本地资源中)) (该效果截图未包含在本地资源中))、右键加 open in code。现存问题: 空 content argument 不让进入 (单纯还没做); 浮层位置不稳定, “点了一下之后要找半天才能关掉” (效果: (该效果截图未包含在本地资源中))。

6.10 LaTeX → Typst 转换

336. **Rust 重写的增量 TeX 引擎实时翻译 demo** (效果: (该效果截图未包含在本地资源中)): 与 mitex 做法不同——后台是完整的用 Rust 重写的增量 TeX 引擎 (区别于 tug2026 那个“增量 latex”: 后者只是渲染层局部增量、后台是常驻 luatex)。原理: 在展开前注入语义标记、再走一遍节点列表把标记和文字配对, 重建语义树交给 Typst 排版; 因为展开是真的, 用户宏和宏包代码可以直接正确处理, 剩余工作量在 LaTeX 命令到 Typst 的映射上 (正文主体可用, 版式细节和 tikz 不行)。目前用 wasm 插件处理所以慢, 直接 fork typst 源码会快很多。TeX 宏自动翻译的边界: 简单宏 (如 `\newcommand{\keyword}[1]{\textbf{#1}} → #strong[]`) 可以, 但 TeX 宏是 token 重写器不是函数, `catcode`、`\@ifnextchar` 前瞻、分隔参数、半展开等在 Typst 没有对应物, 不存在通用自动翻译; 更实际的用途是辅助校验人工移植的 Typst 模板与 LaTeX 原版包行为一致 (差分预言机), 且从展开后的语义树翻译时用户宏、if、计数器都已展开。
337. **浏览器端公式渲染方案讨论 (Typst vs KaTeX/MathML)**: Typst 0.15 支持原生 MathML 输出, 可直接编译公式为 MathML 嵌入 HTML, 无需 WASM/SVG 中转; 但 MathML 在浏览器渲染尴尬——align 只有 Firefox 正常, Chrome/Safari 无法对齐; 且 Typst math 很容易写出“只靠 MathML 不能表达的部分”。成本对比: katex 静态渲染成本 > typst 静态渲染成本; katex 动态渲染成本 < typst 动态渲染成本。能接受 SVG 的话 typst 可直接替代 katex (缺点是公式多时 SVG 总量大)。相关项目: <https://tejasprabhune.github.io/kern/>; <https://ydx-2147483647.github.io/best-of-typst/#math>; 另有项目手写递

归下降 parser 从 typst 源码输出 AST 再处理为 katex/MathML——是互补关系。a11y 思路：公式输出两份，MathML 做无障碍。

338. **Typst** 外部生成图片的需求讨论：Typst 不支持像 LaTeX escape shell 那样调外部命令生成图片（可复现性/安全考虑），讨论过 deno 式权限控制（写文件/exec/网络）的可能性。现有绕过：套一层 make/shell/python 构建；prequery 包 <https://typst.app/universe/package/prequery>；.Rtyp 文档里写 R 代码作图。持久化编译缓存：“好像也有持久化 cache 的 issue，疑似说要做但没人管”——目前增量编译只限于一次 typst 执行之中，冷启动仍耗时，靠 tinymist 或 watch 缓解。

6.11 绘图与曲线算法

339. 自适应采样平面参数曲线：需求源于一条 GeoGebra Curve(...) 定义的复杂参数曲线，局部（尖点附近）需要极大采样密度，等距取样效果差（效果：（该效果截图未包含在本地资源中）（该效果截图未包含在本地资源中）（该效果截图未包含在本地资源中））。让 gemini 抄 GeoGebra 的 curve 实现效果仍差一点；建议方向：tupper 类基于区间算术的绘图算法（结论：区间算术更适合隐式曲线而非参数曲线，且输出是位图，每次编译重跑算法不划算，不如外部画好再引入）；或暴力 10 万个点 + Douglas-Peucker 抽稀。最终代码见 [file:temp.typ]、[file:curve.typ]（GPT 生成），效果（该效果截图未包含在本地资源中）。
340. 图表标签避让算法讨论：输入各标签文字和期望坐标，输出相互避让后的坐标。可行思路：bounding box 都是 AABB 时直接写 AABB 碰撞检测 + while 循环平移直到无冲突；SAT/AABB 只负责检测，避让靠平移。可参考移植 Python 的 <https://pypi.org/project/adjusttext/> 或 <https://pypi.org/project/textalloc/>；高性能方案：Rust 插件里放 HashMap<(u32, u32), Vec<AABB>> 或 RTree。提出者最终选择手动调整（“覆盖复杂场景还是比较困难”）。

6.12 图片与 PDF 兼容性

341. 插入知网 PDF 图片导致 Safari/苹果预览渲染错乱（完整排查记录）：现象：Typst 导出 PDF 在 Safari、苹果预览、快速查看里乱掉（第二张图缩小叠到第一张上），其它阅读器（QQ、Foxit 之外）和导出 PNG 正常；LaTeX \includegraphics[page=N]{image.pdf} 同样出错 → 排除 Typst 实现问题，锁定 image.pdf 本身。进一步：单独插入任意一页都正常，从插入的第二张图开始才出错；给第一个 image 设 height: 50% 时第二个 image 打开瞬间会在正确位置闪现（否则固定在错误位置）。知网原版 PDF（CNKI_ReaderEx 生成的 writer）就有问题；CAJ 转 PDF（<https://www.dpdf.com/zh/caj-to-pdf>）依然不行。最终解法：ghostscript 重写 PDF：

```
gs -o image.fixed.pdf -sDEVICE=pdfwrite -dPDFSETTINGS=/prepress image.pdf
```

```
gs -o image.fixed.pdf -sDEVICE=pdfwrite -dPDFSETTINGS=/prepress image.pdf
```

▲ 代码实测（本机 typst 渲染）

（运行时有一些警告，效果：，AI 解释）论坛帖：<https://forum.typst.app/t/a-pdf-image-corrupts-the-pdf-output-in-ios-safari/9544>。原始 PDF 在 Safari 乱掉样例：，相关文件 [file:PDF 图片乱掉.zip] [file:main.pdf] [file:CAJ2PDF.pdf]。

6.13 编译器机制杂记

342. **LLM** 采样阶段接入增量编译器的可行性讨论：设想让 LLM 在 token 抽样阶段就考虑 Typst 增量编译响应（mask 掉不合语法的 token）。反对意见：线性输出的文档任意截断不一定合法；一秒输出几十 token，编译器跟不上；JSON schema 有此类基础设施但 Typst 语法太复杂；token 采样在 GPU、Typst 编译在 CPU，数据位置造成困难。结论：不如让 LLM 直接接受编译器反馈（typst 编译很快，很现实）。相关论文：<https://arxiv.org/html/2403.01632v1>、<https://arxiv.org/html/2511.22277v2>。
343. 手写手稿风格的实现讨论：仿 X 上 @Sam_Axiom 的手稿效果（https://x.com/Sam_Axiom）：手写字体 + #set page(background: image(...)) 笔记本背景；纸张不平/笔画凹痕效果靠字体实现费事（理论上不依赖具体字迹，可用 metafont 思路）；装饰性 box 建议提前做好 PNG 靠 scale（反对意见：长宽比无法自然调整，直接拉伸不自然，理想是 box 变大时画更多的弧而不是拉伸）。

344. 群 **FAQ** 机器人指令备忘: `/typ <代码>` (渲染)、`/typtyp` (仅高亮)、`/typm` (数学)、`/typ eval` (求值)、`/typdev` (开发版编译, 如 `main.2026-08-04.540db57`)、`/univ <包名>` (查 `universe` 包示例, 如 `tblock/pause` 均未收录)。机器人偶发 `Failed to load typst package registry: error = ConnectError('')`, 不影响无包代码。

7 第六部分 群聊精选（2026-07-01 07-08）

7.1 公式 HTML 导出（MathML / SVG / 性能）

350. **typst** 公式导出 SVG vs MathML 的浏览器差异: firefox > safari > chrome (后两者基本不可用)。inline SVG 是图片、可被 CSS 重着色, 暗黑模式需自行适配; MathML 直接走页面文本色、自动适配但各浏览器行为不一。性能: 300+ 公式时 inline SVG 让 HTML 从 120 KiB 膨胀到 2.6 MiB (约 22×), mathjax 仅 1.1 MiB (zipped 204 KiB); 短文章 inline SVG 优雅, 长论文体建议 MathML 或 epub 分页显示当前页。(来源: YDX 的 mkdocs 指南 forum.typst.app/t/guide-render-typst-math-in-mkdocs)

7.2 语言特性 / 数据结构

351. 复数矩阵乘法 (代码—原理): 群友用“结构体风格”实现 `construct_complex / construct_matrix / mul_matrix`, 完整可编译 3×3 复数乘法 (`get/set_/add/mul` 方法闭包 + 越界 panic)。结论: 数量级小时 **wasm** 调用开销 (**6.90s**) 远大于原生计算 (**246ms**), 小矩阵优先原生或 **wasm**。
352. 变长参数解构 (**first, ..rest**): `numbering` 是多参数函数, (`first, ..rest`) => `numbering("1.1", ..rest)` 把第 2 级起的编号透传; `..rest` 再展开为多个参数。适用于不知道有多少级标题编号时 ([img:待查] 问答场景)。
353. **rect** 内首行缩进 / 段落判定: 要让 `rect()` 内文字受 `par` 影响 (首行缩进、`justify`), 内容须是段落——用 `par[...]` 或加空行; `#rect()[# "两个黄鹂"]` 不行 (inline 文字不算段落), 应 `#rect(par[说的道理])`。(



图 6 聊天中的效果截图

354. **wasm plugin** 不支持多线程: **wasm** 多线程提案尚未定; “无状态”指函数无副作用, 不等于不能多线程 (线程 `join` 有时间差)。经典多线程方案是在状态机里做线程管理, 无状态时该方案不好实现。
355. **show** 规则覆盖顺序陷阱: 后写的 `show` 会覆盖前面的。用户“例题号随一级标题自动计数 + 每章重置”卡住, 根因是计数器更新被后续 `show` 覆盖, 且 `context` 中 `counter(...).update` 会延迟到下次插入元素才生效 (“第一次插入后才更新”是 `expected`)。

7.3 编号 / 计数器

356. 例题号随一级标题自动计数 + 每章重置 (完整可编译):

```
#set heading(numbering: "1.")
#let example-counter = counter("example")
#show heading.where(level: 1): it => { example-counter.update(0); it }
#let example(body) = {
  example-counter.step()
  block(context {
    "例" + numbering("1.", counter(heading).get().first(), example-
counter.get().first())
    linebreak()
    body
  })
}
```

```
}  
= test  
#example[a] // 例1.1
```

要点：读取标题号与例题号都放进 context；理解 update 的延迟效应。（

```
#let example-state = state("example-state", (heading: none, num: 0))  
  
#let example(body) = [  
  #context {  
    let head_num = counter(heading).at(here()).first()  
    example-state.update(prev => {  
      let num = if prev.heading == head_num {prev.num + 1} else {1}  
      (heading: head_num, num: num)  
    })  
  }  
  [  
    #strong[例 #head_num.#example-state.get()]  
    #body  
  ]  
]
```

2.1 等式
例 2.(heading: 1, num: 13)
例 2.(heading: 2, num: 1)
例 2.(heading: 2, num: 2)

图 7 聊天中的效果截图

)

7.4 脚注 / 尾注 / 版面

357. 脚注“跑上一页”是 **known issue**（争议两年）：当前 **typst** 行为“摆前面引用下去，摆后面引用者上去”——脚注若对应下一页的引用，会出现“两页一个 1”。出版业规范：脚注与原文应同页；规避法是提前换行 / 略压缩行距 / 溢出版心（即“孤行控制”思想）。**typst** 目前无配置此规则的选项，**issue** 区未定论。旁注（放右侧）也是合理方案。
358. 超长脚注适合尾注：汉语书超长脚注多为尾注改来；译注脚注 + 原书尾注常见于学术书。**typst** 不靠包实现尾注：把所有注释塞进一个文献条目的 **title** 用 **bibliography** 打印（“相信 **typst** 的脚本能力”）。

7.5 排版 bug / 版本差异

359. 首行缩进被压缩（**PR 8570**, **v0.15.1** 修复）：长文中某段 **overfull**，当“首行以左括号开头”时 **first-line-indent** 量被自动调小（未焊死），导致 **overfull**。减小括号周围绿色 **space** 宽度可缓解；**PR 8570** 把行首左括号左边缘固定到设定值，合并后消失。（

```
#set par(  
  first-line-indent: (amount: 2em, all: true),  
  justify: true,  
)  
#let space = box(width: width, height: 0.8em, fill: green)  
  
(1) 桑之未落，其叶#space;沃若。#linebreak(justify: true)
```

v0.15.1	This PR
(1) 桑之未落，其叶沃若。	(1) 桑之未落，其叶沃若。
(1) 桑之未落，其叶沃若。	(1) 桑之未落，其叶沃若。
(1) 桑之未落，其叶沃若。	(1) 桑之未落，其叶沃若。
(1) 桑之未落，其叶沃若。	(1) 桑之未落，其叶沃若。
(1) 桑之未落，其叶沃若。	(1) 桑之未落，其叶沃若。

图 8 聊天中的效果截图

)

360. **SVG 重影大坑 (XML 实体)**：**typst** 输出 **SVG** 含 **<script>** 里的 ** **，而 ** ** 不是合法 **XML** 实体，用 **image/svg+xml** 的 **DOMParser** 解析失败 → 隐藏文本层 (**.tsel { color: transparent }**) 样式丢失 → 文本层变黑与字形轮廓层叠成重影。根因：** ** 非法 **XML**，应写 **&nbsp;**。排查一晚。

361. **tinymist** 双向跳转定位偏差：点击 `#text("...")` 内字符时游标跳到 `#text` 函数 `span` 而非字符串偏移（字符串无 `span`，`tinymist` 未渲染字符串偏移）。上游问题，非自己编辑器 `bug`。
362. 表格/文本断行怪（**Unicode** 断行算法）：如 `.bashrc` 文件清单断行异常，因 **Unicode** 断行算法不让在特定处换；相关 `pr8750`。可加 `glob/zero-width` 处理。
363. 等宽数学字体：大写 `L/H` 作下标要等宽 → 用 `Courier` 等打字机字体（数学也有等宽的）。

7.6 绘图 / 盒模型

364. 不等宽 **stroke**: `box(stroke: (y: 0.5pt, x: 1pt), radius: 1em, inset: (y:0.3em,x:0.4em))` 可设不同方向不同线宽（“好邪恶”）。（



图9 聊天中的效果截图

365. **stroke/fill** 前景背景层：`typst` 无专门控制谁是前景层；`cetz` 中实现“文字 `stroke` 在背景层”= 一个 `content` 画两遍（先背景层带 `stroke`，再前景层不带 `stroke`）。
366. 注音 **nǎ**：用 `box(stroke: 0.5pt, radius: 1em, inset: (y:0.3em,x:0.4em))[nǎ]` 包带组合环的字符（**Unicode** 组合记号实现）。
367. **organization / hierarchy chart** 包：`typart` 包的 `#tree`；或用 `Fletcher` 自搓。
368. 三线表：`typst-doc-cn` `FAQ` 有 `auto-three-line-table / three-line-table` 页面，可用 `show/set` 把普通表转三线表。

7.7 字体

369. **New Computer Modern** 即 **Typst** 默认数学字体：`Typst` 不会自动搜 `TeX Live` 字体，`app` 用需上传字体文件；本地要有字体。`#set text(font: "Computer Modern")` 需本机装了对应字体。
370. 汉字等宽字体 / 中文衬线术语：`maple mono cn`、`Noto Sans Mono CJK SC` 等宽；中文排版讲“宋体/黑体”而非严格“衬线/无衬线”（楷体可 `serif` 可 `sans`）。

7.8 文献 / bib

371. **bib** 显示 `\textit` 异常：从 zotero 导出的 bib 含 `\textit`（LaTeX 合法）在 `typst bibliography` 不识别（`typst` 写法不同）。解决：换不带 `\textit` 来源、正则 `workaround`、或在 bib 里直接写 `typst` 函数。字体效果像 `ttc` 问题（见 `c00111` / `c01236`）。（

```
1 export-data.bib
2
3 @article{bravo_ingestion_2011,
4   title = {Ingestion of \textit{Lactobacillus} strain regulates emotional
5   volume = {108},
6   issn = {0027-8424, 1091-6490},
7   url = {https://pmc.org/doi/full/10.1073/pnas.1102999108},
8   doi = {10.1073/pnas.1102999108},
9   abstract = {There is increasing, but largely indirect, evidence pointing
10  Lactobacillus rhamnosus
11  could have a direct effect on neurotransmitter receptors in the
12  rhamnosus.
```

参考文献引用测试 [1], [2], [3], [4]

参考文献

- [1] J. A. Bravo et al., "Ingestion of \textit{Lactobacillus} strain regulates emotional behavior and central GABA receptor expression in a mouse via the vagus nerve," *Proceedings of the National Academy of Sciences*, vol. 108, no. 38, pp. 16050–16055, Sep. 2011, doi: 10.1073/pnas.1102999108.
- [2] H. Li et al., "\textit{Lachnospira elgensii} attenuates epileptogenesis via gut-brain axis regulation of blood-brain barrier integrity and neuroinflammation," *Pharmacology*, vol. 16, no. 2, pp. 1045–1062, Jan. 2026, doi: 10.7150/ihno.116959.

图 10 聊天中的效果截图

7.9 工具链 / 生态包

372. 协作网站：collabst（类 `overleaf` 的 `typst` 网站）；TeXlyre（评分更高）；北大 TOSS（纯前端 `typst` 网站）。

373. **numblex** + **numbly** 组合：

```
#import "@preview/sela:0.1.0": sel, any
#import "@preview/numbly:0.1.0": numbly
#import "@preview/numblex:0.1.0": numblex
#let deep = sel(heading.where(level: any(..range(2, 7))))
#set heading(numbering: numblex("{[1]}{[1]}{. [1]}{. [1]}")) // 第1级不编号
#show deep: set heading(numbering: numbly("Chapter {1:A}", "Section {1:A}.{2}",
"Topic {3}."))
```

（注：`#show sel(..)`：直接把函数调用放 `show` 左侧在 0.14 会报 “expected expression”，需先 `#let` 绑定变量；`numblex` 的 `import` 原贴有、此处曾漏。）先全 `set` 再 `show heading.where(level: 1)` 可调换顺序，让单文件日后作为 `chapter` 嵌入大文档时只需恢复一级编号。

374. **pointless-size** 包：临时设字号无需导包，README 有数值可抄。（

typst

Universe

🏠 Home

💰 Pricing

📄 Docs

🗣️ Forum

👤 Sign out

Go to app

🏠 Start

📦 Packages

📁 Templates

🔍 Search

📂 Browse Categories

📤 Submit

📦

pointless-size 0.1.3

中文字号的号数制及字体度量单位 Chinese size system (hào-system) and type-related measurements units

📄 universe 0.1.3

🌟 stars 12

📄 changelog

中文字号的号数制及字体度量单位。Chinese size system (hào-system) and type-related measurements units.

```
#import "@preview/pointless-size:0.1.3": zh, zihao

#set text(size: zh(5)) // 五号 (10.5pt)
// or
#set text(zh(5))
#show: zihao(5)

// 小号用负数或 .5 表示 Use negative numbers or .5 for small sizes
#zh(-4) #zh(4.5) // 小四 (12pt)
#zh(1) #zh(1.8) // 一号 (26pt)
#zh(-1) #zh(1.5) // 小一 (24pt)
#zh("0") #zh(0.5) // 小初 (36pt)
#zh(0) #zh(0.8) // 初号 (42pt)

// 写汉字也可以 Han characters are also acceptable
#zh("五号") #zh("五")
#zh("小五")
```

How to add

Copy this into your project and use the import as pointless-size

#import "@preview/pointless-size:0.1.3" @

Check the docs for more information on how to import packages.

About

Author: Y.D.X.

License: MIT

Current version: 0.1.3

Last updated: August 19, 2026

First released: September 14, 2024

Archive size: 4.06 kB

Homepage: ydx-2147483647.git...

Repository: GitHub 12 0 2-0% open

🕒 2026/8/15

Categories:

📄 Text

🌐 Languages

Explore more packages by Y.D.X.

Where to report issues?

This package is a project of Y.D.X. Report issues on their repository. You can also try to ask for help with this package on the Forum.

Please report this package to the Typst team using the contact form if you believe it is a safety hazard or infringes upon your rights.

Version history

Version	Release Date
0.1.3	August 19, 2026
0.1.2	July 8, 2025
0.1.1	February 3, 2025
0.1.0	September 14, 2024

号数 点数 意义

zh(0)	42pt	初号
zh(1)	26pt	一号
zh(2)	22pt	二号
zh(3)	16pt	三号
zh(4)	14pt	四号
zh(5)	10.5pt	五号
zh(6)	7.5pt	六号
zh(7)	5.5pt	七号
zh(8)	5pt	八号

号数 点数 意义

zh("0")	36pt	小初
zh(1.5)	24pt	小一
zh(2.5)	18pt	小二
zh(3.5)	15pt	小三
zh(4.5)	12pt	小四
zh(5.5)	9pt	小五
zh(6.5)	6.5pt	小六

初 二 三 四 五 六 七 八

小初 小一 小二 小三 小四 小五 小六

覆盖定义 Override

字号没有统一规定，本包默认与 CTeX、MS Word、WPS、Adobe 的中文规则一致。Chinese size systems were not standardized. By default, this package is consistent with Chinese rules of CTeX, MS Word, WPS, Adobe.

如想覆盖定义: If you want to override:

```
#import "@preview/pointless-size:0.1.3": zh as _zh

#let zh = _zh.with(overrides: ((7, 5.25pt),))

#assert.eq(_zh(7), 5.5pt)
#assert.eq(zh(7), 5.25pt)
```

参考资料 References

基本参考链接 Basic Reference Links (中文 + English)

资料汇编: 汉字号数与点数的映射关系 Reference Compilation: Mapping Between the Chinese Size System and the Point Unit (中文 only)

分章版网页 per-chapter homepage (~0.2 MB) / 合集版 all-in-one (~30 MB)

Typst GmbH did not create this package and cannot guarantee correct functionality of this package or compatibility with any version of the Typst compiler or app.

Home

Pricing

Documentation

Universe

About Us

Open Source

Contact Us

Privacy

Terms and Conditions

Legal (Impressum)

Brand Guidelines

Forum

Tools

Blog

GitHub

Discord

Mastodon

Bluesky

LinkedIn

Instagram

Made in Berlin

图 11 聊天中的效果截图

375. **pattern** 早已改名 **tiling**: 老代码 `pattern(...)` 全编译不了 (删于大版本前); `outline` API 在 0.12/0.13 改过 (0.11 用户踩坑, 需看新文档)。

376. **kern** 方案 (tejasprabhune/kern): 网页留 `raw tex + katex.js`, 测试 20% 漏斗率, 效果差强人意。

377. 中文社区 **guide PR**: `typst.dev/guide` 没介绍 Typst 是什么, HackYardo 发 PR 加“快速开始”翻译 + tagline 三方案投票 (A 排版原神启动 / B Markdown LaTeX Office 不顺手 / C 更全能...)。(

47 / 57

Typst 中文社区导航

Markdown、LaTeX、Office 都用不顺手？
不妨试试 Typst——排版原神，启动！

[快速开始](#)[常见问题](#)[我是 Word 用户](#)[发现](#)[介绍](#)[取用](#)[学用](#)[解惑](#)

Typst 是什么？

- **网页办公应用**，可前往 [typst.app/play](#) 直接尝试；当然本地亦可使用
- **标记编程语言**，轻松写作与精细排版两不误，综合 Markdown、LaTeX 所长
- **增量式编译器**，修改文档即刻预览，响应时间为毫秒级至秒级

Typst 能用在哪儿？边界如何？

- 图书、发票、小告示，追求高效自动与成本低廉的商业出版
- 讲义、论文、试卷等，数学公式或交叉引用繁多的教育需求
- 笔记、简历、幻灯片，能套用模板并个人定制的可打印文档
- 读取数据、简单计算、绘图制表能融入文档排版，但并非 Excel 式计算软件

推荐资料

中文社区资料：

- [📍 本站](#)——导航、快速介绍、常见问题解答
- [📘 小蓝书 The Raindrop-Blue Book](#)——入门教程
- [📖 中文文档](#)——官方文档的非官方翻译
- [clreq-gap for typst](#)——分析 Typst 与中文排版的差距
- [可用于 Hayagriva 的 CSL 样式](#)——让 Zotero 中文社区的参考文献样式能被 Typst 接受
- [CSL Validator](#)——克隆 CSL 官方检查器，适配中文 CSL 样式和大陆网络环境
- [泰普斯屯](#)——新闻信息汇总

其它资料：

- [📖 官方文档 Typst Documentation](#)——教程、参考手册、指南（英文）
- [🌐 官方宇宙 Typst Universe](#)——浏览各种包、模板的平台（英文为主，个别页面有中文）
- [Typst Examples Book](#)——有非常多例子（英文）
- [Awesome Typst](#)（英文为主，亦有中文专版）

图 12 聊天中的效果截图



图 13 聊天中的效果截图

378. 讲义模板.typ（群文件）：用户做例题号模板时参考；群里有成熟版本可借。
379. **faq_bot** 资源画像：运行时 CPU 1%、内存 60%、8h、磁盘 85%（typst / typst-dev / 字体 / 包缓存）。
- ### 7.10 其它
380. **typst** 生成二维码：blog.ensko.at/2026/08/11/typst-qr-codes/。
381. **RTL** 网格：figure(text(dir: rtl, grid(columns: 3, gutter: 0.5em, ..range(6).map(n => [#n])))) 实现从右到左排布。
382. **LaTeX** 吐槽梗：“除开编译慢、安装大、语法难、写作难、不直观、文档杂、管理散的缺点之外，总体质量还是很好的”——群内对 LaTeX 的经典调侃。

8 第七部分 群聊精选（2026-07-08 起）

8.1 字符 / 字体 / 颜色

- 400. 获取字符 **unicode** 名称: `typst` 用专属名字, 无内置 `ord` 式函数; 有 `package` 查表 (本质查表)。最快方法是直接查 `unicode` 表。
- 401. 彩色 **emoji** 显示: `text` 默认填充黑色, 导致 **Noto Color Emoji** 预览空白, 但导出 **PDF** 常能显示彩色 (单色版则预览/PDF 都黑白)。结论: 以导出 **PDF** 为准; 需本地装对字体文件 (Color Emoji 矢量版在 `vscode` 预览不显, 单色版可预览)。([img:待查])
- 402. **CMYK** 颜色映射歪: `cmyk` 经“转 `rgb` 再转回”会映射错位, 疑似 `bug` (“已经是 `cmyk` 还歪”)。插件显示颜色与定义不符时优先怀疑颜色空间转换。
- 403. 数学字体不是普通字体: `#show math.equation: it => text(style:"italic", font:("Times New Roman","KaiTi"), it)` 报 `current font is not designed for math`——Times New Roman 非数学字体。以假乱真 Times 用 **TeX Gyre Termes Math / STIX Two Math**。
- 404. `sym.angle` 被 `cetz` 的 `angle` 模块遮蔽 (经典坑): `angle.acute` 报 `type angle does not contain field 'acute'`, 因为 `cetz` 包导出 `angle` 模块, `angle` 被识别为模块而非符号; 正确写 `sym.angle.acute`, 或 `#let angle = sym.angle`。 `angle` 还是 `typst` 类型名, 仅在文件作用域。“之前能编译”是因没 `import cetz`。(

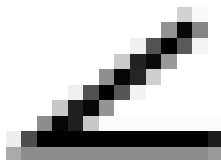


图 14 聊天中的效果截图

图 15 聊天中的效果截图

）补充 c0074 的变体表。

8.2 文字描边 / 阴影 / 行距

- 405. **stroke miter-limit / join** 控制描边尖角: `text(stroke: (thickness: .5em, miter-limit: 2))` 调锐角; `join: "round"` 圆角。字体描边也能改 (不光曲线)。([img:待查])
- 406. 先 **stroke** 再 **fill** 的 **hack** (强制引用顺序等): `#let strk(it) = { box(width:0em, inset:(right:-1000em), text(stroke:(thickness:1.5pt, join:"round"), it)); box(it) }`——把描边层叠在底层。同思路可让“参考文献按指定顺序编号”: 在前面先引用一遍 (`#box(width:0em,...)[...] place hide` 之类 0 宽 0 高 hack)。
- 407. 文本阴影: 官方 FAQ typst.dev/guide/FAQ/text-shadow.html; svg 方法用 `feDropShadow` (有 draft pr, 删 `rect` 无影响)。
- 408. 行距/文字外框超出: `#set text(top-edge:"ascender", bottom-edge:"descender")` 防止中文/特定字体行框下方超出; 外框计算与 `leading` 见 typst.dev/guide/FAQ/par-leading.html。([img:待查])
- 409. **baseline** 顶点对位: `#set text(bottom-edge:"descender", top-edge:"ascender") + box/circle + place(dx:, dy:)` 把小圆点精准落在「回」字绿框顶点 ([img:待查] 问如何落点)。

8.3 数学

- 410. 同余式 **mod**: `$a equiv b (mod m)$` 需手动加空格; 可包一层函数。

8.4 绘图 / 曲线

- 411. 风车旋转中心: `windmill` 用 `box(height: width, width: width)` (注意应是 `2*width` 区域), `place(rotate(blade, 90deg, origin: bottom+right))` 四次; 旋转中心要正中间需 `origin: center+horizon` 且 `box` 尺寸正确。([img:待查])

412. **curve** 超出 **block**: `curve(...)` 含负坐标 `line((200pt,-50pt))` 时部分超出 **block** 边框——负数坐标导致; **block** 用正尺寸即可。
413. **parametric-curve-2d**: 群友实现自适应采样 + 间断点检测 (判据用夹角或三角形面积), 讨论加 NURBS 支持; 采样算法可参考 GeoGebra `curve` / `deepwiki`。
414. 像素化图片: 插入 16×16 png 放大应见清晰像素边界——`typst` 默认最近邻插值 (提问未结, 无显式 `image(..)` 插值开关讨论)。

8.5 布局 / 对齐

415. **h(1fr)** 换行后右对齐: `#h(1fr)` 在“刚好换行位置”后内容会左对齐。要页码永远靠右 (页眉/目录), 把内容塞进 `grid` 让 `1fr` 有东西撑开 (参考证明符号实现)。

8.6 编号 / 文献 (GB/T 7714 续)

416. **omni-gb7714** 完整可编译示例 (自定义 `driver` / 尾注 / `ibid`):

```
#show: gb7714(  
  note: "foot", version: 2015,  
  custom-drivers: (book: "creator{, }title{, }publisher{, }date{, p.}pages"),  
  custom-terms: (ibid: (text: "Ibid.", supplement-separator: ", pp.")),  
)  
// note: "end" 出尾注; note-ibid: false 复刻 2025 CSL; footnote-number 前缀可定制  
#bibliography(bytes("@book{zhang2019, ...}"))
```

`note: "end"` 时尾注表在 `#bibliography` 调用处打印。(

前面一段话, 第一次引用这本书^①。中间再写一句, 紧接着又引同一本书的
另一处^②。

参考文献

[1] 张三, 《中国古代城市规划史》, 商务印书馆, 2019

^①张三, 《中国古代城市规划史》, 商务印书馆, 2019, p.42

^②Ibid., pp.114-514

图 16 聊天中的效果截图

第一次引这本书^①。这里引另一本^②。回头再引第一本的另一处^③, 紧接着又
引它一次^④。

参考文献

[1] 张三. 中国古代城市规划史[M]. 北京: 商务印书馆, 2019.

[2] 李四. 市政制度考[M]. 北京: 中华书局, 2020.

^①张三. 中国古代城市规划史[M]. 北京: 商务印书馆, 2019: 42.

^②李四. 市政制度考[M]. 北京: 中华书局, 2020.

^③见注^①, 114-514.

^④见注^①, 7.

图 17 聊天中的效果截图

第一次引这本书^①。这里引另一本^②。回头再引第一本的另一处^③，紧接着又引它一次^④。

参考文献

[1] 张三. 中国古代城市规划史[M]. 北京：商务印书馆，2019.
[2] 李四. 市政制度考[M]. 北京：中华书局，2020.

^①张三. 中国古代城市规划史[M]. 北京：商务印书馆，2019：42.
^②李四. 市政制度考[M]. 北京：中华书局，2020.
^③见注^①，114-514.
^④同上：7.

图 18 聊天中的效果截图

②李四. 市政制度考[M]. 北京：中华书局
③可以随意定制前缀①!!!! 114-514.
④同上：7

图 19 聊天中的效果截图

第一次引这本书^①。这里引另一本^②。回头再引第一本的另一处^③，紧接着又引它一次^④。

以上是正文，
以下是尾注：

① 张三. 中国古代城市规划史[M]. 北京：商务印书馆，2019：42.
② 李四. 市政制度考[M]. 北京：中华书局，2020.
③ 可以随意定制前缀①!!!! 114-514.
④ 同上：7.

图 20 聊天中的效果截图

)

417. 脚注带页码 **@key[p. 3]**：原生 gb-7714-note 第一个 p.3 不显示（CSL 缺 locator 变量，需在 ibid 分支复制）；hayagriva500 因 CSL 测试集与 citeproc.js 绑定而被屏蔽。p./pp. 单复数目前只能手动，或正则 hack 进自定义 bib 字段。（

孔乙己¹ 上大人²

参考文献

[1] AUTHOR. Title[Z]. 2025.

¹AUTHOR. Title[Z]. 2025.

²同上: p. 4.

图 21 聊天中的效果截图

)

8.7 工具链 / 插件

418. **typst** 内“代码—效果”对照（极简版 **FAQ** 机器人原理）：

```
#let code = raw(block: true, lang: "typ", "你的代码")
#code
#eval(code.text, mode: "markup")
```

你的代码

你的代码

▲ 代码实测（本机 **typst** 渲染）

`#raw(block: true, lang: "typ", code) + #eval(code, mode: "markup")` 即可在文档里自含代码示例与渲染结果（原贴用围栏直接嵌 **raw** 块，嵌套围栏时需加长外层定界符）。

419. **tinymist** 事实：vscode 插件导出 PDF 用内置 **typst**；**tinymist** 自身不支持导出 **png/svg**（用 F1 “导出为特定格式”）；支持命令行（`winget install tinymist`），可被 **copilot** 智能体调用。预览未保存文档可用 **Typst Ultra** 插件。

420. **metalogo** 包：`#import "@preview/metalogo:1.2.0": TeX, LaTeX` 输出正规 logo。

421. 模板制作 = 会写函数 + **return** + **show**：中文书籍模板群里有 **modern-nju-thesis**、**uwnibook-color** 等：“自己弄费时间，有人花个把月做模板肯定比我强”。

422. 隐水印 **package** 思路：`content` → 渲染 **png**（需 **typst-as-wasm-plugin**）→ 高频处理 → **png** → **page background**；更务实是写构建文件（**typstmk** 之类）。

8.8 语言陷阱 / AI 协作

423. 裸内容逃逸（**raw content escape**）：AI 生成的右圆括号关闭 **#table**，后续内容被当普通文本裸输出（无报错）。防御：`#show "": panic("...")`（但会搞乱其它）；本质是 AI 错误 + 标记语言二义。务实做法：让 AI 再扫一遍 / 人眼校 / **formatter**。`#{ ... }` 比直接写 `[]` 更安全。

424. **typst** 代码块自动折行丢缩进：PDF 阅读器复制/编译时自动折行把缩进丢了，**Python** 代码乱。务实：`show raw.where(block: true): set text(0.9em)`；折行行距应比普通小以便区分（

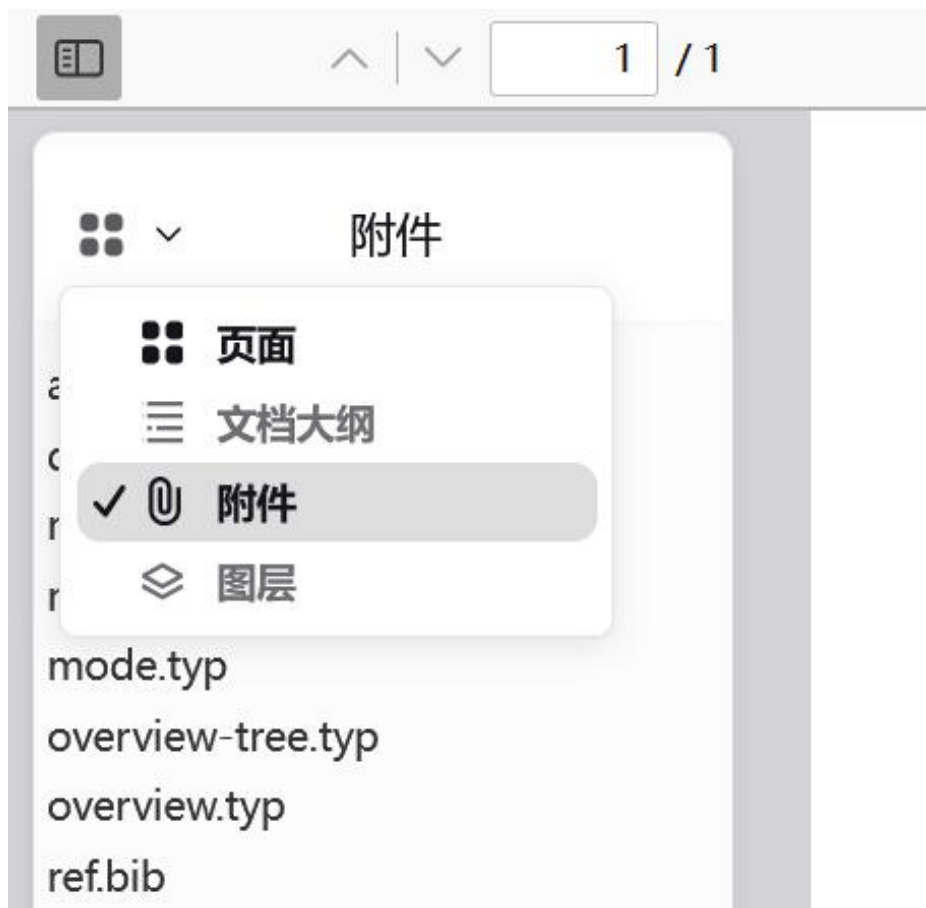


图 22 聊天中的效果截图

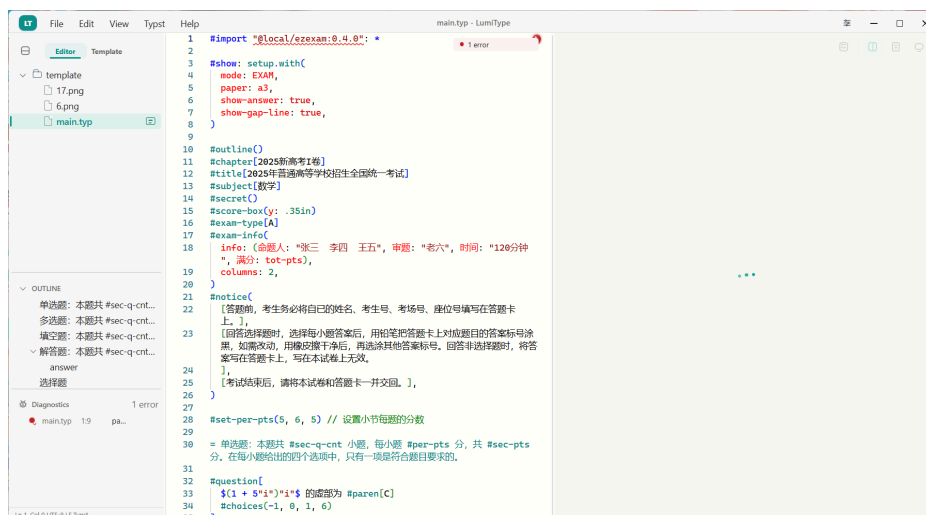


图 23 聊天中的效果截图

)。不同阅读器 (UPDF/Edge/Firefox) 行为不一。

425. **sym.angle** 模块冲突 见 404。

426. **float 除零 vs float.inf**: $1.0/0.0$ 报 cannot divide by zero (整数除零也报), 而 `calc.exp(1e16)` 返回 `float.inf`。防除零加 `epsilon`, 但需兼顾 `float.inf/float.nan` (754 的 $0/\text{nan}$ 不止一个, 是经典 `float vulnerability pattern`)。typst 非科学计算语言, 采样分母为 0 概率高时加 `epsilon` 是常规解法。

427. **helix 复制 sampled values**: `metadata(x) / text(font: x, "") / [#label(x)]` 从 typst 命令行输出复制 (不便, 但可行)。(

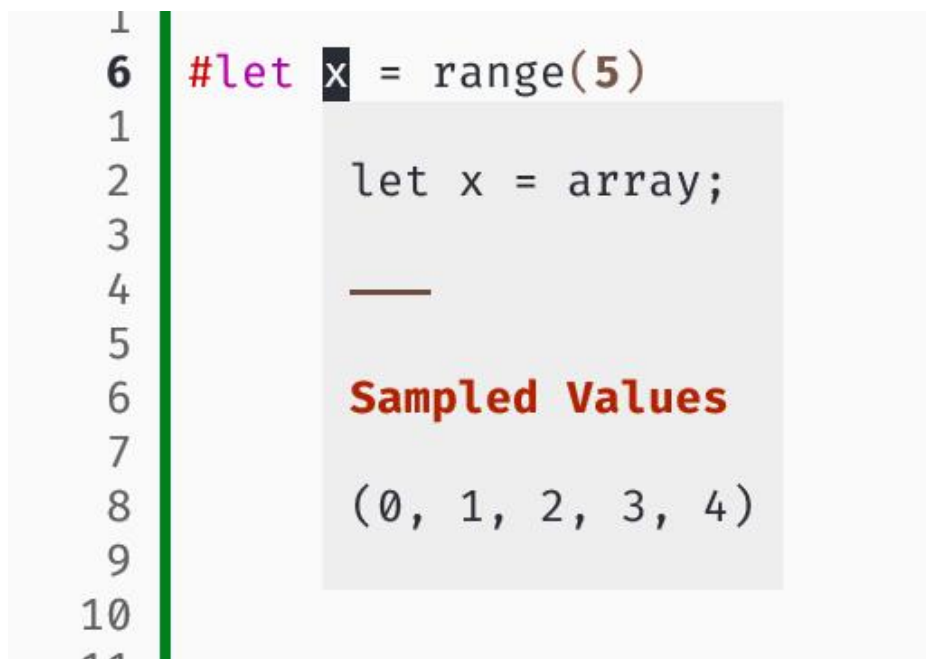


图 24 聊天中的效果截图

)

8.9 导出 / 多端

- 428. **HTML 幻灯片 + view transition:** typst 导出 html 搭配 View Transition API 平滑动画（或 Web Animations）；难点是“用于演讲的 html 与用于分发的 pdf 一模一样”——直接传 PDF 才 100% 一致（issue8309）。
- 429. 竖排不支持：typst 本身不支持竖排；蒙文/满文这类从右往左换行 + 拆 cluster 旋转性能极差。阿文社区讨论比蒙文多。

8.10 包 / 资源

- 430. 群文件：tennis-parabola.typ（抛物线例题）、a.typ/a.pdf（AI 复现评测）、marker-prototypes-multilingual.pdf/marker-font-comparison-multilingual.pdf、Qwen3.8-27B-2026-09-03T08.typ/Qwen3.8-27B-2026-09-03T08.pdf（本地小模型复现 PDF 实测）、讲义模板.typ（见 c03378）。
- 431. **AI 复现 PDF 的共性翻车：**link 被转成 text(rgb(...))、部分加粗变 list/enum、表格格式错乱、页码用 page.footer 实现导致全居左（应居中或左右相间）；中日期号位置差异导致 align(center) 的四语言行未对齐（需逐行 box 套住单独对齐）。

